第 1 讲：程序执行、复杂度与从思路到代码

1. 本讲目标

完成本讲后，你应该能够：

- 按顺序模拟一段 Python 代码的执行；
- 区分变量、状态、循环和条件；
- 判断常见单层、双层和折半循环的复杂度；
- 理解时间复杂度不是精确运行时间；
- 把一个简单思路细化成可以编码的步骤；
- 发现空数组、负数、重复值等基本边界问题。

这一讲是后面所有数据结构的基础。你当前的主要困难不是完全没有思路，而是思路还没有具体到能够直接翻译为代码。

---

2. 程序是怎样执行的

计算机不会理解“找一下最大的数字”这种概括。它只能执行明确的小步骤。

观察下面的代码：

```
[python]
nums = [4, 1, 7, 3]
ans = nums[0]

for x in nums[1:]:
    if x > ans:
        ans = x

print(ans)
```

执行过程可以写成状态表：

当前 `x`	循环开始时 `ans`	`x > ans`	循环结束时 `ans`
1	4	False	4
7	4	True	7
3	7	False	7

这里的 `ans` 表示：

> 截至目前已经看过的元素中的最大值。

这句话叫作变量的“含义”，也可以理解为循环过程中始终想维持的事实。以后写算法时，先说清一个变量代表什么，代码会容易很多。

2.1 四个最基本的组成部分

许多简单算法都可以拆成：

1. **状态**：需要记住的信息；
2. **初始化**：开始时状态是什么；
3. **更新**：处理一个新元素后，状态如何变化；
4. **结果**：什么时候返回、返回什么。

对“找最大值”来说：

```
          ans
ans = nums[0]
  x > ans   ans = x
        ans
```

2.2 为什么 `ans = 0` 可能错误

下面的实现无法正确处理全部为负数的数组：

```
[python]
def find_max(nums):
    ans = 0
    for x in nums:
        if x > ans:
            ans = x
    return ans
```

例如：

```
[python]
nums = [-8, -3, -10]
```

函数会返回 `0`，但 `0` 根本不在数组中。

一个更合理的实现是：

```
[python]
def find_max(nums):
    if not nums:
        return None

    ans = nums[0]

    for x in nums[1:]:
        if x > ans:
            ans = x

    return ans
```

这里先单独处理空数组，因为空数组不存在 `nums[0]`。

注意：真实项目中也可以对空数组抛出异常。LeetCode 通常会在题目中规定输入是否可能为空。面试时如果题意没有说明，应主动确认。

---

3. 从自然语言到代码

“我知道怎么做”可能有不同程度。

3.1 不够具体的思路

还缺少的信息包括：

- 怎样判断以前见过这个元素？

- 需要保存什么？
- 找到后返回什么？
- 没找到时返回什么？

3. 2 可以编码的思路

```
seen
   x
x     seen      True
 x    seen
           False
```

3. 3 翻译为 Python

```
[python]
def contains_duplicate(nums):
    seen = set()

    for x in nums:
        if x in seen:
            return True
        seen.add(x)

    return False
```

这里值得注意两个位置：

1. 必须先检查 `x in seen`，再执行 `seen.add(x)`；
2. `return False` 必须在循环结束后，而不是循环内部。

错误示例：

```
[python]
def contains_duplicate(nums):
    seen = set()

    for x in nums:
        seen.add(x)
        if x in seen:
            return True
```

这个版本永远会在第一个元素处返回 `True`，因为检查之前已经把当前元素放进了集合。

另一个错误示例：

```
[python]
def contains_duplicate(nums):
    seen = set()

    for x in nums:
        if x in seen:
            return True
        seen.add(x)
        return False
```

这个版本最多只检查第一个元素。第一次循环结束前就返回了 `False`。

3. 4 编码前的六个问题

对简单题，可以固定问：

```
1.
2.
3.
4.
```

5.
6.

如果这些问题都有明确答案，代码通常只是翻译。

---

4. 什么是时间复杂度

时间复杂度描述：

> 当输入规模 `n` 增大时，算法执行步骤的数量以什么速度增长。

它不是精确的秒数，也通常不计算每一个常数步骤。

4.1 `O(1)`：固定数量的操作

```
[python]
def first_element(nums):
    return nums[0]
```

无论 `nums` 有 10 个还是 10 万个元素，都只访问一次，因此是 `O(1)`。

`O(1)` 不一定只执行一行。只要执行次数不随着 `n` 增长，就可以看作 `O(1)`。

4.2 `O(n)`：遍历一次

```
[python]
def total(nums):
    ans = 0
    for x in nums:
        ans += x
    return ans
```

数组有 `n` 个元素，循环执行 `n` 次，所以是 `O(n)`。

下面仍然是 `O(n)`：

```
[python]
for x in nums:
    print(x)

for x in nums:
    print(x * 2)
```

总步骤大约是 `2n`。复杂度分析忽略常数倍，因此：

```
O(2n) = O(n)
```

4.3 `O(n²)`：对每个元素再次遍历

```
[python]
for x in nums:
    for y in nums:
        print(x, y)
```

外层执行 `n` 次，每次内层再执行 `n` 次，总计 `n × n`，所以是 `O(n²)`。

两数之和的暴力解法也是 `O(n²)`：

```
[python]
def two_sum_brute_force(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
```

虽然内层循环不是每次都完整执行 `n` 次，但比较总数仍然与 n^2 同一个增长等级。

4.4 嵌套循环不一定都是 $O(n^2)$

不要看到两个 `for` 就机械判断为 $O(n^2)$。

```
[python]
for row in matrix:
    for value in row:
        print(value)
```

如果整个矩阵一共有 N 个元素，每个元素只访问一次，那么复杂度相对于元素总数是 $O(N)$。

关键问题是：最内层语句总共执行了多少次？

4.5 $O(\log n)$：每次缩小一半

```
[python]
n = 100

while n > 1:
    n //= 2
```

变化过程：

```
100 → 50 → 25 → 12 → 6 → 3 → 1
```

每次都把问题规模缩小一半，因此循环次数大约是 $\log_2 n$，复杂度为 $O(\log n)$。

二分查找是最典型的 $O(\log n)$ 算法。

4.6 $O(n \log n)$

高效的通用排序通常是 $O(n \log n)$。现阶段先记住常见大小关系：

```
O(1) < O(log n) < O(n) < O(n log n) < O(n²)
```

当 $n = 100000$ 时，大致可以感受为：

复杂度	操作数量级
$O(1)$	固定少量
$O(\log n)$	约 17 次折半
$O(n)$	约 10 万
$O(n \log n)$	约 170 万
$O(n^2)$	约 100 亿

这不是精确运行时间，只用于感受增长速度。

---

5. 分析复杂度的基本规则

5.1 忽略常数

```
O(3n) → O(n)
O(100) → O(1)
```

5.2 只保留增长最快的项

```
O(n² + n + 10) → O(n²)
```

当 `n` 很大时，`n²` 起主导作用。

5.3 连续执行通常相加

```python
[python]
for x in nums:       # O(n)
    print(x)

for y in nums:       # O(n)
    print(y)
```

结果为 `O(n + n) = O(n)`。

5.4 真正嵌套通常相乘

```python
[python]
for x in nums:       # n
    for y in nums:   #    n
        print(x, y)
```

结果为 `O(n × n) = O(n²)`。

5.5 提前返回不改变最坏情况

```python
[python]
def contains(nums, target):
    for x in nums:
        if x == target:
            return True
    return False
```

如果目标恰好是第一个元素，执行一次即可。但最坏情况下目标不存在，需要检查所有元素，因此最坏时间复杂度是 `O(n)`。

面试中如果没有特别说明，通常分析最坏时间复杂度。

---

6. 什么是空间复杂度

空间复杂度描述算法运行时额外使用的空间怎样随输入规模增长。

6.1 `O(1)` 额外空间

```python
[python]
def total(nums):
    ans = 0
    for x in nums:
        ans += x
    return ans
```

不论数组多长，只使用少数变量，因此额外空间是 `O(1)`。

输入数组本身一般不算算法额外创建的空间。

6.2 `O(n)` 额外空间

```python
[python]
def copy_numbers(nums):
    result = []
    for x in nums:
        result.append(x)
    return result
```

`result` 的长度随着 `n` 增长，因此额外空间为 `O(n)`。

使用集合记录所有已见元素也可能需要 `O(n)` 空间：

```python
seen = set()
for x in nums:
    seen.add(x)
```

6.3 时间与空间的交换

判断重复元素可以用双重循环：

- 时间：`O(n²)`；
- 额外空间：`O(1)`。

也可以使用集合：

- 时间：平均 `O(n)`；
- 额外空间：`O(n)`。

这叫空间换时间。面试中经常需要说明这种取舍。

---

7. 常见边界条件

写完主逻辑后，至少检查：

7.1 空输入

```python
nums = []
```

是否访问了不存在的 `nums[0]`？

7.2 只有一个元素

```python
nums = [5]
```

循环是否正确？应该返回什么？

7.3 全部为负数

```python
nums = [-7, -2, -9]
```

最大值初始值是否错误地设成 `0`？

7.4 重复值

```python
nums = [3, 3]
```

题目要求返回值、下标还是布尔值？

7.5 答案在开头或结尾

边界位置容易暴露循环范围错误。

7.6 根本没有答案

循环结束后是否有明确返回？

---

8. 手动调试方法

如果代码结果不对，不要只靠反复修改。选一个最小反例，记录关键变量。

例如：

```
[python]
nums = [4, 2, 4]
```

对判断重复的算法记录：

当前 `x`	判断前 `seen`	是否已存在	操作
4	{}	False	加入 4
2	{4}	False	加入 2
4	{2, 4}	True	返回 True

手动跟踪能够帮助你发现：

- 更新顺序错误；
- `return` 缩进错误；
- 循环少执行或多执行一次；
- 状态含义和实际内容不一致。

---

9. 本讲必须记住的 Python 写法

遍历元素

```
[python]
for x in nums:
    print(x)
```

遍历下标

```
[python]
for i in range(len(nums)):
    print(i, nums[i])
```

同时获得下标和元素

```
[python]
for i, x in enumerate(nums):
    print(i, x)
```

创建和更新集合

```
[python]
seen = set()
seen.add(5)

if 5 in seen:
    print("    ")
```

提前返回

```
[python]
for x in nums:
    if x < 0:
        return True

return False
```

注意最后的 `return False` 位于循环外部。

---

10. 本讲总结

1. 计算机需要明确、逐步、可执行的指令。
2. 写代码前先确定状态、初始化、更新方式和返回条件。
3. 时间复杂度描述输入增大时操作数量的增长速度。
4. 单次遍历通常是 `O(n)`，真正的双重遍历通常是 `O(n²)`，每次折半通常是 `O(log n)`。
5. 空间复杂度关注额外使用的空间。
6. 能看懂解答不等于能够主动生成代码。
7. 练习时应从空白开始，并用小例子手动跟踪变量。

学完后，请关闭本讲，独立完成练习文件。不要先打开参考答案。

第 2 讲：数组、Python `list` 与原地修改

0. 本讲的问题

假设有一个列表：

```
[python]
nums = [3, 2, 2, 3, 4]
```

现在要求删除所有的 `3`，但有两个限制：

1. 不要创建一个与输入同样长的新列表；
2. 函数需要直接修改原列表的前半部分。

这不是单纯的“会不会删除元素”。它要求我们回答：

> 当从左向右扫描时，怎样让已经处理过的区间始终保持正确？

这个问题会引出本讲最重要的编码工具：**读指针、写指针和循环不变量**。

---

1. 数组是什么

从抽象层面看，数组是一串按顺序排列、可以通过整数下标访问的槽位：

```
  0   1   2   3
[8] [5] [9] [2]
```

访问 `nums[2]` 时，程序不需要先查看 `nums[0]` 和 `nums[1]`，因此按下标读取或修改通常是 `O(1)`：

```
[python]
x = nums[2]     #
nums[2] = 100   #
```

1.1 Python `list` 更准确地说是动态数组

C 的普通数组通常具有固定长度。Python 的 `list` 可以执行：

```
[python]
nums.append(7)
```

长度能够增长，所以它更接近“动态数组”。

从实现直觉上，可以把 Python `list` 想成一排保存对象引用的槽位。它不是 C 中那种直接紧密存放同类型整数的普通数组，因此：

- 元素可以是不同类型；
- 列表保存的是对象引用；
- 列表自身可以扩容；
- 访问第 `i` 个槽位仍然是 `O(1)`。

本课程中说“数组题”时，通常使用 Python `list` 来完成。

---

2. 为什么 append 平均是 O(1)

如果列表后面还有预留空间，append 只需要把新元素放进下一个槽位，因此是 O(1)。

如果空间用完，Python 需要申请更大的区域，并复制已有引用。某一次扩容可能是 O(n)。

但它不会每添加一个元素就扩容一次，而是一次多申请一些容量。假设容量大致按比例增长，连续添加 n 个元素时，复制量类似：

$$1+2+4+8+\cdots < 2n$$

因此，n 次 append 的总成本仍然是 O(n)，平均到每一次就是 O(1)。这种分析称为**摊还复杂度**。

你不需要记住 Python 的实际扩容比例，只要记住：

> list.append 是摊还 O(1)，偶尔一次扩容会更慢。

---

3. 常见操作的复杂度

设列表长度为 n：

操作	常见复杂度	原因
nums[i]	O(1)	直接定位槽位
nums[i] = x	O(1)	直接修改槽位
len(nums)	O(1)	长度被单独保存
nums.append(x)	摊还 O(1)	通常在末尾直接加入
nums.pop()	O(1)	删除末尾元素
nums.insert(0, x)	O(n)	后面的元素需要移动
nums.pop(0)	O(n)	后面的元素需要前移
x in nums	O(n)	最坏需要逐个比较
nums.index(x)	O(n)	最坏需要逐个查找
nums[a:b]	O(b-a)	创建并复制切片
nums.copy()	O(n)	创建浅拷贝
nums.sort()	O(n log n)	对原列表排序
sorted(nums)	O(n log n)	创建新列表并排序

“原地排序”和“返回新列表”不同：

```[python]
nums.sort()           #    nums    None
result = sorted(nums)  # nums      result
```

---

4. 什么时候遍历值，什么时候遍历下标

4.1 只需要元素值

```
[python]
for x in nums:
    print(x)
```

适用于求和、计数、查找等。

4.2 需要下标

```
[python]
for i in range(len(nums)):
    nums[i] *= 2
```

适用于修改原列表、访问相邻元素、维护多个位置。

4.3 同时需要下标和值

```
[python]
for i, x in enumerate(nums):
    print(i, x)
```

不要为了显得简洁而强行选择某种写法。应根据算法需要选择。

---

5. 下标区间：编码错误的高发区

Python 常用左闭右开区间：

```
[left, right)
```

意思是包含 `left`，不包含 `right`。

```
[python]
range(2, 5)   #    2, 3, 4
nums[2:5]     #      2, 3, 4
```

左闭右开的好处是区间长度直接等于：

\[right-left \]

5.1 常用范围

```
[python]
range(len(nums))              # 0   n-1
range(1, len(nums))           # 1   n-1
range(len(nums) - 1)          # 0   n-2
range(len(nums) - 1, -1, -1)  # n-1   0
```

5.2 相邻元素

如果代码访问 `nums[i + 1]`，就必须保证 `i + 1 < len(nums)`：

```
[python]
for i in range(len(nums) - 1):
    print(nums[i], nums[i + 1])
```

每当代码出现 `i-1` 或 `i+1`，都要重新检查循环边界。

---

6. 切片不是免费的

```
[python]
part = nums[1:4]
```

`part` 是一个新列表，元素来自原列表的下标 `1 2 3`。假设切片长度为 `k`：

- 时间复杂度：`O(k)`；
- 额外空间复杂度：`O(k)`。

所以：

```
[python]
for x in nums[1:]:
    ...
```

虽然循环思想可能只需要 `O(1)` 额外状态，但 `nums[1:]` 会额外创建长度约为 `n` 的列表。

如果需要严格的 `O(1)` 额外空间，可以写：

```
[python]
for i in range(1, len(nums)):
    x = nums[i]
```

切片非常实用，不是不能使用；重点是知道它会复制。

---

7. 引用、别名与浅拷贝

观察：

```
[python]
a = [1, 2, 3]
b = a
b[0] = 100

print(a)  # [100, 2, 3]
```

`a` 和 `b` 指向同一个列表。`b = a` 没有复制列表，只是增加了一个名字。

如果希望得到一个新的外层列表：

```
[python]
b = a.copy()
#
b = a[:]
```

但两者都是浅拷贝。对于嵌套列表：

```
[python]
a = [[1, 2], [3, 4]]
b = a.copy()
b[0][0] = 99

print(a)  # [[99, 2], [3, 4]]
```

外层列表不同，但内部的子列表仍然是共享对象。

7.1 一个经典陷阱

```
[python]
grid = [[0] * 3] * 3
grid[0][0] = 1
```

结果是：

```
[python]
```

```
[[1, 0, 0],
 [1, 0, 0],
 [1, 0, 0]]
```

因为三个位置引用的是同一个内部列表。

正确创建独立行：

```
[python]
grid = [[0] * 3 for _ in range(3)]
```

---

8. 原地修改是什么意思

原地修改通常表示：

> 直接改变输入对象，而不是返回一个替代它的新对象。

下面不是原地修改调用者持有的列表：

```
[python]
def clear_numbers(nums):
    nums = []
```

这里只是让函数内部的名字 `nums` 指向新列表，外面的原列表没有改变。

下面会修改原列表：

```
[python]
def clear_numbers(nums):
    nums.clear()
```

切片赋值也会保留原列表对象：

```
[python]
nums[:] = [1, 2, 3]
```

这在题目要求“修改输入数组”时很重要。

---

9. 核心推导：删除指定值

问题：把所有不等于 `val` 的元素移动到列表前面，并返回有效元素数量 `k`。

例如：

```
nums = [3, 2, 2, 3, 4], val = 3
```

最后只要求：

```
k = 3
nums[:k] == [2, 2, 4]
```

`k` 之后的内容不重要。

9.1 暴力想法的问题

可以反复执行 `nums.pop(i)`，但中间删除会让后面的元素移动，而且遍历过程中列表长度发生变化，很容易跳过元素。最坏时间也可能达到 `O(n²)`。

9.2 将问题改写为“保留谁”

与其删除 `val`，不如把所有不等于 `val` 的元素依次写到前面。

使用两个下标：

- `read`：当前正在检查的位置；
- `write`：下一个应该写入有效元素的位置。

循环开始时维持：

> `nums[0:write]` 恰好包含此前读到的所有有效元素，并保持原顺序。

代码：

```
[python]
def remove_value(nums, val):
    write = 0

    for read in range(len(nums)):
        if nums[read] != val:
            nums[write] = nums[read]
            write += 1

    return write
```

9.3 手动执行

输入：

```
nums = [3, 2, 2, 3, 4], val = 3
```

read	当前值	是否保留	写入位置	更新后 write	已确定区间
0	3	否	—	0	`[]`
1	2	是	0	1	`[2]`
2	2	是	1	2	`[2, 2]`
3	3	否	—	2	`[2, 2]`
4	4	是	2	3	`[2, 2, 4]`

结束时，所有元素都已读取，根据不变量，`nums[:write]` 就是全部有效元素，所以返回 `write`。

9.4 复杂度

- 每个元素读取一次：`O(n)` 时间；
- 只增加固定数量变量：`O(1)` 额外空间。

这里的两个指针并不是真正的 C 指针，只是两个保存下标的整数变量。

---

10. 为什么不应边遍历边删除

危险的写法：

```
[python]
nums = [1, 2, 2, 3]

for x in nums:
    if x == 2:
        nums.remove(x)
```

删除第一个 `2` 后，后一个 `2` 会向前移动，但循环的内部位置继续前进，于是可能跳过它。

如果目的只是创建过滤后的新列表，可以写：

```
[python]
result = []

for x in nums:
    if x != 2:
        result.append(x)
```

如果要求原地操作，使用读写指针更可靠。

---

11. 第二个编码模型：从两端向中间

反转列表时，可以交换两端：

```
[1, 2, 3, 4, 5]
 ↑           ↑
left       right
```

交换后，同时向中间移动：

```
[python]
def reverse_in_place(nums):
    left = 0
    right = len(nums) - 1

    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1
```

循环不变量可以描述为：

> `left` 左边和 `right` 右边的元素，已经位于反转后的正确位置。

每次循环至少确定两个位置，因此时间复杂度是 `O(n)`，额外空间是 `O(1)`。

---

12. 写数组代码前的检查表

```
1.
2.
3.
4.    i+1   i-1
5.
6.
7.           O(n)
8.
9.
10.
```

---

13. 本讲总结

1. Python `list` 是动态数组，支持 `O(1)` 下标访问。
2. 末尾添加是摊还 `O(1)`，头部或中间插入、删除通常是 `O(n)`。
3. 切片会创建新列表，不是免费的。
4. `b = a` 只创建别名；`a.copy()` 创建浅拷贝。

5. 原地修改要求改变原对象，而不是只让局部变量指向新对象。
6. 读写指针适合稳定地筛选、覆盖和压缩数组。
7. 两端指针适合反转或利用有序性缩小搜索范围。
8. 写代码时要明确循环不变量：数组的哪一段已经满足什么性质。

接下来打开 Notebook 练习。核心题是“移动零”和“有序数组的平方”，它们会要求你把本讲的两个编码模型迁移到新问题中。

第 3 讲：哈希思想、哈希表、`set` 与 `dict`

0. 本讲的核心问题

给定一串数据，我们经常要回答：

如果每次都在线性列表中查找，单次查询最坏是 `O(n)`。如果一边遍历，一边做 `n` 次这样的查询，总时间可能变成 `O(n²)`。

哈希表的目标是：

> 通过键本身快速推断它应该存放在哪里，使插入、查询和删除在平均情况下接近 `O(1)`。

Python 的 `set` 和 `dict` 都主要建立在哈希表上：

```
├── set
└── dict      →
```

---

1. 从直接寻址开始

假设键只可能是 `0` 到 `999` 的整数。可以准备一个长度为 1000 的数组：

```
[python]
present = [False] * 1000
present[42] = True
```

查询 `42` 是否出现：

```
[python]
present[42]
```

时间是 `O(1)`，因为键本身就是下标。

但如果键可能是：

```
10^18
"alice@example.com"
(-3, 8)
```

就不能为所有可能的键准备槽位。我们需要把一个很大的键空间压缩到有限数量的槽位中。

---

2. 哈希函数与槽位

哈希函数把键转换成整数：

$$ h(key) \rightarrow \text{integer} $$

再根据表的容量映射到一个槽位：

$$ index=h(key)\bmod m $$

其中 `m` 是槽位数量。

直觉示意：

```
"cat"  ──hash──> 18493 ──mod m──>    3
"dog"  ──hash──> 72011 ──mod m──>    7
```

查询时再次执行相同计算，因此不必从头扫描全部元素。

Python 会负责具体的哈希、槽位布局、扩容和冲突处理。做题时我们主要使用它提供的接口。

---

3. 哈希冲突为什么必然存在

可能的键远多于槽位，根据抽屉原理，不同键必然可能映射到同一槽位：

```
key_a ──>    5
key_b ──>    5
```

这叫哈希冲突。

发生冲突不代表两个键相等。哈希表仍需要比较实际的键，并用额外规则寻找或保存它们。常见策略包括链地址法和开放寻址法；Python 的具体实现细节会随版本演进，本课程不要求复现。

你需要掌握的结论：

- 哈希值相同，键不一定相等；
- 相等的键必须拥有相同的哈希值；
- 冲突越多，查询通常越慢；
- 表过满时需要扩容；
- 因此查询、插入、删除通常说“平均 `O(1)`”。

理论最坏情况可能退化，但常规 LeetCode 分析采用平均复杂度。

---

4. 为什么键必须可哈希

键放入哈希表后，系统依赖它的哈希值再次找到原位置。如果键在存储期间发生改变，哈希值也可能改变，原来的位置就失去意义。

因此，Python 哈希表的键通常必须是不可变、可哈希的对象。

通常可以作为键：

```[python]
42
"hello"
(3, 5)
```

通常不能作为键：

```[python]
[3, 5]       # list
{3, 5}       # set
{"x": 3}    # dict
```

元组只有在内部元素也都可哈希时才能作为键：

```[python]
(1, 2)          #
(1, [2, 3])     #
```

二维坐标经常用元组作键：

```[python]
```

```
visited = set()
visited.add((row, col))
```

---

5. `set`：只保存键

5.1 常用操作

```
[python]
seen = set()

seen.add(5)
seen.add(8)
seen.add(5)        #         5

5 in seen          # True
9 not in seen      # True

seen.remove(5)     #       KeyError
seen.discard(5)    #
```

平均复杂度：

操作	平均时间
`x in seen`	`O(1)`
`seen.add(x)`	`O(1)`
`seen.remove(x)`	`O(1)`

集合自身可能保存 `n` 个元素，所以额外空间通常是 `O(n)`。

5.2 集合运算

```
[python]
a = {1, 2, 3}
b = {2, 3, 4}

a & b   #    {2, 3}
a | b   #    {1, 2, 3, 4}
a - b   #    {1}
```

做题时不要只因为存在集合运算符就使用它。仍要分析操作需要遍历多少元素、是否创建新集合。

---

6. `dict`：保存键到值的映射

6.1 常用操作

```
[python]
positions = {}
positions[10] = 3
positions[20] = 7

10 in positions       #     True
positions[10]         #     3
positions.get(99)     #        None
positions.get(99, -1) #        -1
```

注意：

```
[python]
x in positions
```

检查的是键，不是值。

6.2 遍历字典

```python
for key in positions:
    print(key)

for value in positions.values():
    print(value)

for key, value in positions.items():
    print(key, value)
```

现代 Python 的 `dict` 保留插入顺序，但它不是自动按键排序的。算法正确性不应误依赖“看起来刚好有序”。

---

7. 高频模式一：是否出现过

问题：判断数组是否存在重复元素。

```python
def contains_duplicate(nums):
    seen = set()

    for x in nums:
        if x in seen:
            return True
        seen.add(x)

    return False
```

循环不变量：

> 处理当前 `x` 之前，`seen` 恰好包含此前已经处理的元素。

必须先检查、后加入。如果先加入，当前元素一定已经存在。

- 平均时间：`O(n)`；
- 额外空间：`O(n)`。

---

8. 高频模式二：频率统计

问题：统计每个字符出现次数。

```python
def character_counts(text):
    counts = {}

    for ch in text:
        counts[ch] = counts.get(ch, 0) + 1

    return counts
```

这一句可以拆开理解：

```python
old_count = counts.get(ch, 0)
new_count = old_count + 1
counts[ch] = new_count
```

循环不变量：

> 处理位置 `i` 之前，`counts` 正确统计了 `text[:i]` 中每个字符的出现次数。

Python 还提供：

```
[python]
from collections import Counter
counts = Counter(text)
```

学习阶段建议先能手写普通 `dict` 版本，再使用 `Counter`。

---

9. 高频模式三：键映射到附加信息

两数之和

给定 `nums` 和 `target`，寻找：

$$ nums[i]+nums[j]=target,\quad i<j $$

处理当前值 `x=nums[j]` 时，需要的值唯一确定：

$$ needed=target-x $$

如果只使用 `set`，能知道 `needed` 是否出现，却不知道它的下标。于是使用 `dict`：

```
[python]
def two_sum(nums, target):
    positions = {}

    for j, x in enumerate(nums):
        needed = target - x

        if needed in positions:
            return [positions[needed], j]

        positions[x] = j
```

循环不变量：

> 处理下标 `j` 前，`positions` 保存了此前元素的“值 → 某个下标”。

为什么必须先查询、后写入？考虑：

```
[python]
nums = [3, 3]
target = 6
```

在处理第一个 `3` 时不能用它与自己配对。先查询此前元素，天然保证返回的下标小于当前下标。

---

10. 高频模式四：把对象变成规范键

问题：判断两个字符串是否为字母异位词。

```
"anagram"   "nagaram"
```

一种思路是统计频率；另一种思路是把每个字符串转化为规范形式：

```
[python]
key = "".join(sorted(word))
```

所有由相同字母组成的字符串会得到相同的键。

分组异位词：

```
[python]
def group_anagrams(words):
    groups = {}

    for word in words:
        key = "".join(sorted(word))

        if key not in groups:
            groups[key] = []

        groups[key].append(word)

    return list(groups.values())
```

这里的设计问题不是“是否使用字典”，而是：

> 什么键能够准确代表一个分组？

如果单词平均长度为 `k`，排序每个单词需要 `O(k log k)`，总时间约为 `O(nk log k)`。

---

11. 进阶模式：最长连续序列

给定无序数组：

```
[100, 4, 200, 1, 3, 2]
```

最长连续序列是：

```
1, 2, 3, 4
```

长度为 4。

如果从每个数字都向后搜索，会重复工作。例如从 `1 2 3 4` 分别出发。

关键观察：只有当 `x-1` 不存在时，`x` 才可能是一个序列的起点。

```
[python]
def longest_consecutive(nums):
    values = set(nums)
    best = 0

    for x in values:
        if x - 1 not in values:
            current = x
            length = 1

            while current + 1 in values:
                current += 1
                length += 1

            best = max(best, length)

    return best
```

虽然有嵌套的 `while`，总平均时间仍是 `O(n)`：只有序列起点会启动向后搜索，每个连续元素只在其所属序列中被访问一次。

这再次说明：不能只数循环层数，要计算最内层操作总共执行多少次。

---

12. 常见错误

12.1 把值和下标混淆

```
[python]
for i, x in enumerate(nums):
    positions[i] = x       #    →
```

两数之和通常需要：

```
[python]
positions[x] = i           #   →
```

写字典前先用一句话说出“键表示什么，值表示什么”。

12.2 更新顺序错误

同一个结构中，“先查后写”和“先写后查”可能产生不同结果。必须根据不变量决定顺序。

12.3 使用不可哈希的键

需要把列表状态作为键时，通常转换为元组：

```
[python]
key = tuple(values)
```

12.4 忘记空间代价

把列表转换为集合：

```
[python]
values = set(nums)
```

平均查询更快，但额外空间是 `O(n)`。

12.5 误以为字典自动排序

保留插入顺序不等于按键大小排序。

---

13. 选择 `set` 还是 `dict`

先问：

> 除了“这个键是否存在”，我还需要保存什么信息吗？

需求	结构
是否出现过	`set`
去重	`set`
值对应下标	`dict`
元素出现次数	`dict`
分类键对应一组元素	`dict`
坐标是否访问过	`set[tuple]`

---

14. 本讲编码检查表

```
1.
2.      dict
3.
4.
5.
6.
7.
8.
9.
10.
```

15. 本讲总结

1. 哈希表通过键的哈希值快速定位存储位置。
2. 冲突不可避免，所以查询复杂度通常表述为平均 `O(1)`。
3. `set` 是只保存键的哈希集合；`dict` 保存键和值。
4. 键必须可哈希，列表、集合和字典通常不能直接作为键。
5. 高频模式包括：存在性、频率、值到下标、规范键分组。
6. 解题的核心仍然是定义状态和不变量，而不是机械地“看到数组就套哈希表”。

第 4 讲：栈——保存尚未完成的工作

0. 栈解决什么问题

考虑括号字符串：

```
([{}])
```

读到最后一个 `)` 时，它应该与哪个左括号匹配？

答案不是最早出现的 `(` 以外的任意括号，而是**最近一个尚未匹配的左括号**。

这种“最近加入的未完成任务最先处理”的规则就是：

```
Last In, First Out
```

栈不仅用于括号。它常常保存：

- 尚未匹配的符号；
- 尚未计算的操作数；
- 尚未完成的函数调用；
- 等待未来元素解决的问题；
- 撤销操作需要的历史状态。

---

1. 栈是规则，不是一种固定语法

栈是一种抽象数据类型，它规定可以怎样操作数据：

- `push(x)`：把元素压入栈顶；
- `pop()`：删除并返回栈顶元素；
- `peek()`：查看栈顶但不删除；
- `is_empty()`：判断是否为空。

栈可以用动态数组实现，也可以用链表实现。

Python 通常直接使用 `list`：

```
[python]
stack = []

stack.append(10)  # push
stack.append(20)

top = stack[-1]   # peek    20
x = stack.pop()   # pop       20
```

`append()` 和末尾 `pop()` 都是摊还或通常 `O(1)`。

不要使用 `pop(0)` 模拟栈，因为删除开头需要移动后面的元素，是 `O(n)`。

---

2. 栈的状态图

执行：

```
[python]
stack = []
stack.append("A")
stack.append("B")
stack.append("C")
stack.pop()
```

状态变化：

```
[]
[A]
[A, B]
[A, B, C]
[A, B]       # C
```

只能直接操作顶部，并不意味着不能读取 Python 列表中的其他位置；而是算法主动遵守栈的访问规则。

---

3. 用类封装一个栈

Python 已经可以用 `list` 当栈，但手写一个小类有助于理解 `class` 和 `self`：

```
[python]
class ArrayStack:
    def __init__(self):
        self._items = []

    def push(self, value):
        self._items.append(value)

    def pop(self):
        if not self._items:
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self):
        if not self._items:
            raise IndexError("peek from empty stack")
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0

    def __len__(self):
        return len(self._items)
```

创建两个对象：

```
[python]
a = ArrayStack()
b = ArrayStack()

a.push(1)
b.push(9)
```

`a._items` 和 `b._items` 属于不同对象。调用：

```
[python]
a.push(1)
```

Python 自动把 `a` 传给 `self`。

`_items` 前的单下划线表示“这是内部实现细节”的约定，并不会真正禁止外部访问。

---

4. 模式一：匹配最近的未完成元素

4.1 有效括号

规则：

- 左括号出现时，还不能判断它与谁匹配，因此压栈；
- 右括号出现时，它必须匹配栈顶；
- 最后栈必须为空。

```
[python]
def is_valid_parentheses(text):
    pairs = {
        ")": "(",
        "]": "[",
        "}": "{",
    }
    stack = []

    for ch in text:
        if ch in "([{":
            stack.append(ch)
        else:
            if not stack:
                return False

            if stack.pop() != pairs[ch]:
                return False

    return not stack
```

循环不变量：

> 处理当前位置之前，`stack` 从底到顶保存了此前尚未匹配的左括号；栈顶是最先需要被未来右括号匹配的那个。

4.2 为什么要先判断空栈

输入：

```
")"
```

如果直接 `stack.pop()` 会产生 `IndexError`。这不是语法问题，而是没有先确认算法状态允许弹栈。

4.3 为什么结尾还要检查空栈

输入：

```
"((("
```

遍历过程中没有遇到错误右括号，但仍有左括号未匹配，所以必须：

因此函数最后应执行 `return not stack`。

---

5. 模式二：栈保存可合并的中间结果

删除相邻重复字符

输入：

```
"abbaca"
```

过程：

```
a      -> [a]
b      -> [a,b]
b      ->          -> [a]
a      ->          -> []
c      -> [c]
a      -> [c,a]
```

结果为 `"ca"`。

```
[python]
def remove_adjacent_duplicates(text):
    stack = []

    for ch in text:
        if stack and stack[-1] == ch:
            stack.pop()
        else:
            stack.append(ch)

    return "".join(stack)
```

不变量：

> 处理完当前前缀后，`stack` 是这个前缀完全消除相邻重复字符后的结果。

注意条件顺序：

条件应写成 `if stack and stack[-1] == ch:`。

Python 的 `and` 会短路。只有 `stack` 非空时才计算 `stack[-1]`。

---

6. 模式三：表达式求值

逆波兰表达式把运算符放在操作数之后：

```
["2", "1", "+", "3", "*"]
```

对应：

\[(2+1)\times3=9 \]

读到数字时压栈；读到运算符时弹出两个操作数，计算后把结果压回。

```
[python]
def evaluate_rpn(tokens):
    stack = []

    for token in tokens:
        if token not in {"+", "-", "*", "/"}:
            stack.append(int(token))
            continue

        right = stack.pop()
        left = stack.pop()

        if token == "+":
            value = left + right
        elif token == "-":
            value = left - right
        elif token == "*":
            value = left * right
```

```
            else:
                value = abs(left) // abs(right)
                if (left < 0) != (right < 0):
                    value = -value

            stack.append(value)

        return stack[-1]
```

减法和除法的顺序非常重要。后弹出的才是左操作数：

```
left operator right
```

LeetCode 的该题要求除法向零截断。先对绝对值整除，再根据符号决定正负，可以避免浮点数参与。Python 直接对负数使用 `//` 是向下取整，语义不同：

```
[python]
-3 // 2 == -2
int(-3 / 2) == -1
```

---

7. 设计题：支持常量时间最小值的栈

普通栈可以 `O(1)` 压入和弹出，但如果每次用 `min(stack)` 查询最小值，需要 `O(n)`。

解决方法：为每个入栈元素同时保存“到这里为止的最小值”。

```
  5   2   4   1
5   2   2   1
```

可以用两个栈：

```
[python]
class MinStack:
    def __init__(self):
        self.values = []
        self.minimums = []

    def push(self, value):
        self.values.append(value)

        if not self.minimums:
            self.minimums.append(value)
        else:
            self.minimums.append(min(value, self.minimums[-1]))

    def pop(self):
        self.minimums.pop()
        return self.values.pop()

    def top(self):
        return self.values[-1]

    def get_min(self):
        return self.minimums[-1]
```

两个栈长度始终相同。`minimums[i]` 表示 `values[:i+1]` 的最小值。

所有操作都是 `O(1)`，但使用了 `O(n)` 额外空间。

也可以在一个栈中保存 `(value, current_min)`，原理相同。

---

8. 单调栈：保存还没找到答案的元素

问题：对每一天的温度，求还要等几天才会出现更高温度。

```
temperatures = [73, 74, 75, 71, 69, 72, 76, 73]
answer       = [ 1,  1,  4,  2,  1,  1,  0,  0]
```

当读到温度 `76` 时，它会同时解决此前多个尚未找到更高温度的日子。

栈中保存下标，而不是温度值，因为答案需要计算距离：

```
[python]
def daily_temperatures(temperatures):
    answer = [0] * len(temperatures)
    stack = []

    for today, temperature in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < temperature:
            previous_day = stack.pop()
            answer[previous_day] = today - previous_day

        stack.append(today)

    return answer
```

循环结束时，栈中剩余下标右侧都没有更高温度，答案保持 `0`。

8.1 为什么是单调栈

从栈底到栈顶，对应温度保持非递增。任何比当前温度低的栈顶都会被弹出，直到重新满足这个性质。

8.2 为什么不是 $O(n^2)$

虽然 `for` 内部有 `while`，但每个下标：

- 只入栈一次；
- 最多出栈一次。

所以所有 `while` 执行次数加起来最多是 `n`，总时间为 `O(n)`。

这与最长连续序列一样：不能只看循环嵌套层数，要计算元素在整个算法中被处理的总次数。

---

9. 调用栈与递归

函数调用本身也使用栈。调用新函数时，系统保存：

- 返回到哪里；
- 局部变量；
- 当前执行状态。

递归不断压入新的调用帧，返回时再逐层弹出。递归过深会触发 Python 的递归深度限制。

以后学习树的 DFS 时，会看到：

- 递归版本使用系统调用栈；
- 迭代版本显式使用 Python 列表作为栈。

它们在结构上是相似的。

---

10. 什么时候想到栈

题目出现以下信号时，可以考虑栈：

但不要因为题目中有“顺序”就使用栈。核心是是否符合后进先出。

---

11. 常见错误

11.1 对空栈执行 `pop` 或读取 `[-1]`

```
[python]
if stack and stack[-1] == value:
    ...
```

11.2 弹出顺序错误

对于减法、除法等不满足交换律的运算，先弹出的是右操作数。

11.3 栈中保存了错误的信息

每日温度需要距离，所以保存下标；只保存温度无法直接计算相隔天数。

11.4 只在局部维护性质，没有检查最终状态

括号题遍历完还要确认栈为空。

11.5 把栈当队列

- 栈：末尾加入、末尾取出；
- 队列：一端加入、另一端取出。

队列将在下一讲使用 `collections.deque`。

---

12. 本讲检查表

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.

13. 本讲总结

1. 栈是后进先出的访问规则，可以用 Python `list` 实现。
2. 栈经常保存“最近的尚未完成工作”。
3. 括号匹配、相邻消除和表达式求值是基本模式。
4. MinStack 通过同步保存历史最小值，把查询最小值降到 `O(1)`。

5. 单调栈保存尚未找到答案的元素，并维持单调性质。

6. 判断复杂度时，应计算每个元素总共入栈、出栈多少次。

第 5 讲：队列、双端队列与 BFS

0. 本讲的核心问题

假设火从若干位置同时向上下左右扩散，每分钟前进一格。怎样计算每个位置最早在第几分钟被烧到？

这个问题的关键不是“递归”或“枚举所有路径”，而是：

> 距离起点近的状态必须先处理，距离远的状态后处理。

这种先到先处理的规则就是队列；按照距离一层层扩展的搜索就是广度优先搜索 BFS。

---

1. 队列是什么

队列遵循：

```
First In, First Out
```

基本操作：

- `enqueue(x)`：从队尾加入；
- `dequeue()`：从队首取出；
- `front()`：查看队首；
- `is_empty()`：判断是否为空。

状态示意：

```
A  [A]
B  [A, B]
C  [A, B, C]
   [B, C]       # A
```

栈处理最近加入的任务，队列处理最早等待的任务。

---

2. Python 为什么使用 `deque`

使用普通列表：

```
[python]
queue = []
queue.append(x)
x = queue.pop(0)
```

虽然逻辑正确，但 `pop(0)` 会让后面的元素整体前移，时间是 `O(n)`。

应使用：

```
[python]
from collections import deque

queue = deque()
queue.append(10)       #
queue.append(20)
x = queue.popleft()    #         10
```

`deque` 是双端队列，两端加入和删除通常都是 `O(1)`：

```
[python]
queue.append(x)
queue.appendleft(x)
queue.pop()
queue.popleft()
```

但 `deque` 不适合频繁通过中间下标随机访问。数组擅长随机访问，双端队列擅长两端操作。

---

3. 用类理解队列接口

```
[python]
from collections import deque

class Queue:
    def __init__(self):
        self._items = deque()

    def enqueue(self, value):
        self._items.append(value)

    def dequeue(self):
        if not self._items:
            raise IndexError("dequeue from empty queue")
        return self._items.popleft()

    def front(self):
        if not self._items:
            raise IndexError("front from empty queue")
        return self._items[0]

    def is_empty(self):
        return not self._items
```

在 LeetCode 中通常直接使用 `deque`，不需要每题重新封装类。

---

4. BFS 的基本思想

给定一个起点：

```
0
1
2
3
```

BFS 使用队列保证距离较小的状态先被处理：

```
[python]
from collections import deque

def bfs(start):
    queue = deque([start])
    visited = {start}

    while queue:
        current = queue.popleft()

        for neighbor in neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

核心不变量：

> 队列中状态按它们被发现的先后顺序排列；在无权图中，状态第一次被发现时，所用步数最少。

5. 为什么访问标记要在入队时设置

推荐写法：

```python
if neighbor not in visited:
    visited.add(neighbor)
    queue.append(neighbor)
```

不要等到出队时才标记。假设 `A` 和 `B` 都连接到 `C`：

```
A —> C <— B
```

如果 `C` 入队时没有标记，处理 `A` 和 `B` 时可能把 `C` 重复加入队列。

入队即标记表示：

> 一个状态一旦被安排处理，就不再重复安排。

6. BFS 求无权图最短距离

图用邻接表表示：

```python
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"],
}
```

计算起点到各点的最短边数：

```python
from collections import deque

def shortest_distances(graph, start):
    queue = deque([start])
    distance = {start: 0}

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in distance:
                distance[neighbor] = distance[node] + 1
                queue.append(neighbor)

    return distance
```

这里 `distance` 同时承担两个作用：

- 判断是否访问过；
- 保存最短距离。

因此不需要额外的 `visited`。

6.1 为什么第一次发现就是最短距离

BFS 先处理距离 `d` 的所有节点，再处理距离 `d+1` 的节点。一个节点第一次被发现时，不可能存在尚未处理的更短路径，否则它应该在更早的一层已经被发现。

这个结论只直接适用于每条边代价相同的情况。带不同权重的最短路不能机械使用普通 BFS。

---

7. 怎样按层处理

有时不仅需要遍历，还需要知道当前层。

```
[python]
from collections import deque

def bfs_by_level(start):
    queue = deque([start])

    while queue:
        level_size = len(queue)
        current_level = []

        for _ in range(level_size):
            node = queue.popleft()
            current_level.append(node)

            for neighbor in neighbors(node):
                queue.append(neighbor)

        print(current_level)
```

必须在处理本层之前保存：

```
[python]
level_size = len(queue)
```

因为处理过程中还会把下一层加入队列。当前这 `level_size` 个元素恰好属于同一层。

二叉树层序遍历会使用这个模板。

---

8. 网格也是图

二维网格中的每个格子可以看作节点，上下左右相邻格子是边。

```
[python]
directions = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1),
]
```

从 `(row, col)` 得到相邻坐标：

```
[python]
for dr, dc in directions:
    nr = row + dr
    nc = col + dc
```

边界检查：

```
[python]
0 <= nr < rows and 0 <= nc < cols
```

8.1 网格最短路模板

```
[python]
from collections import deque

def shortest_path(grid, start, target):
    rows = len(grid)
    cols = len(grid[0])
    queue = deque([(start[0], start[1], 0)])
    visited = {start}
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        row, col, distance = queue.popleft()

        if (row, col) == target:
            return distance

        for dr, dc in directions:
            nr = row + dr
            nc = col + dc

            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] == 1:
                continue
            if (nr, nc) in visited:
                continue

            visited.add((nr, nc))
            queue.append((nr, nc, distance + 1))

    return -1
```

这里假设 `1` 是障碍，无法到达时返回 `-1`。

---

9. 多源 BFS

如果火从多个位置同时开始扩散，不应该分别运行多次 BFS。可以把所有起点一开始同时入队：

```
[python]
queue = deque()

for source in sources:
    queue.append(source)
    visited.add(source)
```

之后普通 BFS 就会自然按照“距离最近的任意起点”扩散。

9.1 腐烂的橘子

网格中：

- `0`：空格；
- `1`：新鲜橘子；
- `2`：腐烂橘子。

每分钟腐烂橘子感染相邻新鲜橘子：

```
[python]
from collections import deque

def oranges_rotting(grid):
    rows = len(grid)
```

```
    cols = len(grid[0])
    queue = deque()
    fresh = 0

    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == 2:
                queue.append((row, col, 0))
            elif grid[row][col] == 1:
                fresh += 1

    minutes = 0
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        row, col, minute = queue.popleft()
        minutes = max(minutes, minute)

        for dr, dc in directions:
            nr = row + dr
            nc = col + dc

            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2
                fresh -= 1
                queue.append((nr, nc, minute + 1))

    return minutes if fresh == 0 else -1
```

把新鲜橘子改成 `2` 就同时完成了访问标记，避免重复入队。

---

10. BFS 的复杂度

图有 `V` 个节点、`E` 条边。使用邻接表时：

- 每个节点最多入队一次；
- 每条边最多被检查固定次数。

因此：

```
O(V + E)
O(V)
```

对于 `rows × cols` 网格，节点数是 `rows × cols`，每格只有常数个邻居：

```
O(rows × cols)
O(rows × cols)
```

---

11. BFS 与 DFS 的区别

特征	BFS	DFS
核心结构	队列	栈或递归
搜索方式	一层层扩展	沿一条路径深入
无权最短路	天然适合	通常不能直接保证
内存特点	可能保存整层	主要保存当前路径
常见题型	最少步数、层序、扩散	连通性、路径、回溯

如果只问“能否到达”，两者都可能适用；如果问“最少几步”，优先考虑 BFS。

---

12. 常见错误

12.1 使用 `list.pop(0)`

逻辑正确但可能把整体复杂度拖慢。使用 `deque.popleft()`。

12.2 出队时才标记访问

可能让同一状态重复入队。通常应入队时标记。

12.3 忘记边界检查顺序

必须先确认坐标合法，再读取 `grid[nr][nc]`。

12.4 混淆当前层与下一层

按层处理时，先保存 `level_size`。

12.5 对带权图机械使用 BFS

普通 BFS 保证的是边数最少，或每条边权重相同情况下的最短距离。

12.6 忘记空网格约束

访问 `grid[0]` 前，要根据题目保证或自行处理空输入。

---

13. 本讲检查表

```
1.
2.
3.
4.
5.
6.
7.
8.              BFS
9.
10.
```

14. 本讲总结

1. 队列遵循先进先出，Python 使用 `deque` 实现高效两端操作。
2. BFS 用队列按照距离由近到远扩展。
3. 无权图中，节点第一次被发现时通常得到最短距离。
4. 已访问状态通常在入队时标记。
5. 多个起点同时扩散时，把全部起点一开始入队。
6. 图 BFS 时间通常是 `O(V+E)`，网格 BFS 通常是 `O(rows×cols)`。

第 6 讲：链表——通过引用连接数据

本地交互讲义：打开 @@CODE0@@。网页为单个本地文件，不需要联网或登录。

0. 本讲的核心问题

数组通过下标访问元素，但在开头插入一个元素时，后面的元素通常都要移动。

能否让元素不依赖连续位置，而是让每个元素指出“下一个元素在哪里”？

这就是链表：

```
[  3 | next] → [  7 | next] → [  9 | None]
```

链表题的难点通常不是数学，而是：

> 修改引用之前，必须确保仍然能找到后续节点。

---

1. 节点与单链表

Python 中常见节点定义：

```
[python]
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

创建链表：

```
[python]
third = ListNode(9)
second = ListNode(7, third)
head = ListNode(3, second)
```

结构：

```
head
 ↓
 3 → 7 → 9 → None
```

`head` 不是整条链表本身，而是对第一个节点的引用。通过不断读取 `.next` 才能访问后续节点。

空链表表示为：

```
[python]
head = None
```

LeetCode 通常已经提供 `ListNode`，你只需要编写操作链表的方法。

---

2. 链表与数组的区别

操作	动态数组 `list`	单链表
按下标访问第 `i` 个	`O(1)`	`O(i)`
已知节点后插入	通常涉及移动	`O(1)`
已知前驱后删除	通常涉及移动	`O(1)`
查找某个值	`O(n)`	`O(n)`
内存组织	逻辑上连续槽位	节点通过引用连接

链表的 `O(1)` 插入有一个前提：已经拿到正确位置的节点。如果还要先遍历寻找位置，总时间仍可能是 `O(n)`。

---

3. 遍历链表

```
[python]
def linked_values(head):
    result = []
    current = head

    while current is not None:
        result.append(current.val)
        current = current.next

    return result
```

循环不变量：

> `current` 指向下一个尚未处理的节点，`result` 保存此前节点的值。

常见简写：

常见简写是 `while current:`。

它与 `while current is not None` 在这里等价。

不要写：

不要写成 `while current.next:`。

这会漏掉最后一个节点，并且空链表时访问 `None.next` 会报错。

---

4. 在已知节点后插入

原结构：

```
A → C
```

插入 `B`：

```
[python]
b = ListNode("B")
b.next = a.next
a.next = b
```

结果：

```
A → B → C
```

顺序很重要。先让 `B` 指向原来的 `C`，再让 `A` 指向 `B`。

如果先执行：

```
[python]
a.next = b
```

而没有提前保存或连接 `C`，就可能丢失原后继节点。

---

5. 删除下一个节点

原结构：

```
A → B → C
```

如果已经拿到 `A`，删除 `B`：

```
[python]
a.next = a.next.next
```

变成：

```
A ———→ C
```

Python 中如果没有其他引用指向 `B`，它最终会由垃圾回收处理。与 C 不同，这里不需要手动 `free`。

---

6. 虚拟头节点 dummy

删除或合并链表时，头节点经常是特殊情况。

例如删除所有等于 `target` 的节点：

```
1 → 1 → 2 → 3
```

如果开头的节点需要删除，`head` 本身会变化。虚拟头节点把头部操作转化为普通的“删除下一个节点”：

```
[python]
def remove_elements(head, target):
    dummy = ListNode(0, head)
    current = dummy

    while current.next is not None:
        if current.next.val == target:
            current.next = current.next.next
        else:
            current = current.next

    return dummy.next
```

为什么删除后不移动 `current`?

因为新的 `current.next` 也可能等于 `target`，需要继续检查。

循环不变量：

> `current` 之前的链表已经处理完毕且不包含目标值；`current.next` 是下一个待检查节点。

---

7. 核心题：反转链表

原链表：

```
1 → 2 → 3 → None
```

目标：

```
None ← 1 ← 2 ← 3
                ↑
              new head
```

7.1 为什么需要三个引用

处理 `current` 时，要把它的 `.next` 改成前一个节点。但修改之前必须保存原来的下一个节点，否则剩余链表会丢失。

```
[python]
def reverse_list(head):
    previous = None
    current = head

    while current is not None:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node

    return previous
```

四步必须理解：

```
1.      next_node = current.next
2.      current.next = previous
3. previous
4. current        next_node
```

7.2 循环不变量

每轮开始时：

- `previous` 是已经反转部分的头节点；
- `current` 是尚未处理部分的头节点；
- 两部分合起来包含原链表全部节点，没有丢失节点。

循环结束时 `current is None`，说明未处理部分为空，`previous` 就是完整反转链表的头。

7.3 复杂度

- 每个节点处理一次：`O(n)` 时间；
- 只使用固定数量引用：`O(1)` 额外空间。

---

8. 合并两个有序链表

```
list1: 1 → 3 → 5
list2: 2 → 4 → 6
```

每次选择两个当前节点中较小的一个接到结果尾部。

```
[python]
def merge_two_lists(list1, list2):
    dummy = ListNode()
    tail = dummy

    while list1 is not None and list2 is not None:
        if list1.val <= list2.val:
            tail.next = list1
            list1 = list1.next
        else:
```

```
                tail.next = list2
                list2 = list2.next

            tail = tail.next

        if list1 is not None:
            tail.next = list1
        else:
            tail.next = list2

        return dummy.next
```

不变量：

> `dummy.next` 到 `tail` 是已经确定的有序前缀；`list1` 和 `list2` 分别指向两个链表尚未合并的第一个节点。

尾部剩余链表已经有序，可以整体接上，不需要逐个复制节点。

时间 `O(m+n)`，额外空间 `O(1)`，因为复用了原节点。

---

9. 快慢指针：寻找中点

让：

- `slow` 每次走一步；
- `fast` 每次走两步。

当 `fast` 到达末尾时，`slow` 走了大约一半。

```
[python]
def middle_node(head):
    slow = head
    fast = head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

    return slow
```

偶数个节点时，这个版本返回第二个中间节点：

```
1 → 2 → 3 → 4
        ↑
        3
```

循环条件顺序很重要：只有确认 `fast` 非空，才能读取 `fast.next`。

---

10. 快慢指针：判断环

有环链表：

```
1 → 2 → 3 → 4
        ↑       ↓
        └───────┘
```

如果快慢指针进入环：

- `slow` 每轮走 1 步；
- `fast` 每轮走 2 步；

- 快指针相对慢指针每轮接近 1 步。

在有限长度的环中，它们最终必然相遇。

```
[python]
def has_cycle(head):
    slow = head
    fast = head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

        if slow is fast:
            return True

    return False
```

这里使用：

```
[python]
slow is fast
```

因为要判断是否为同一个节点对象，而不是节点值是否相同。不同节点完全可以保存相同的 `val`。

- 时间：`O(n)`；
- 额外空间：`O(1)`。

也可以使用 `set` 保存访问过的节点，时间 `O(n)`、空间 `O(n)`。

---

11. 链表相交

两条链表可能共享后半部分节点：

```
A1 → A2 ┐
        ├→ C1 → C2
B1 → B2 ┘
```

相交指的是引用同一个节点对象，不是值相等。

一种巧妙方法：两个指针各自到达末尾后，转去另一条链表的头部。

```
[python]
def get_intersection_node(head_a, head_b):
    a = head_a
    b = head_b

    while a is not b:
        a = head_b if a is None else a.next
        b = head_a if b is None else b.next

    return a
```

若两链表长度为 `m n`，两个指针都走过 `m+n` 的对齐路径，最终在交点或共同的 `None` 相遇。

---

12. Python 引用语义中的常见误解

12.1 改变局部变量不等于改变原链表

```
[python]
def skip_head(head):
    head = head.next
```

调用者的变量不会自动改为第二个节点。函数需要返回新的头：

```
[python]
def skip_head(head):
    return head.next
```

12.2 修改节点字段会影响共享结构

```
[python]
head.next = None
```

这是修改节点对象本身，所有引用到该节点的代码都会观察到变化。

12.3 值相等不代表节点相同

```
[python]
node1.val == node2.val
```

不能证明 `node1 is node2`。

---

13. 常见错误

13.1 修改 `.next` 前没有保存后继

可能丢失剩余链表。

13.2 返回旧的 `head`

反转后新头是最后处理的节点，即 `previous`。

13.3 空节点上访问 `.next`

快慢指针条件应为：

快慢指针循环条件应写成 `while fast is not None and fast.next is not None:`。

13.4 删除后错误地前进

删除连续目标节点时，前驱不应立即前进。

13.5 用值判断相交或成环

应比较节点对象身份。

13.6 只画节点，不画引用变量

调试时应同时画出：

```
previous current next_node head tail
```

分别指向哪个节点。

---

14. 本讲检查表

```
1.
2.    next
3.
4.      dummy
5.              next
6.
7.
8.
9.
10.
```

15. 本讲总结

1. 链表节点通过 `.next` 引用连接，不能通过下标随机访问。
2. 修改引用顺序非常重要，必要时先保存后继节点。
3. 虚拟头节点可以统一处理头部插入和删除。
4. 反转链表需要维护已反转部分与未处理部分。
5. 快慢指针可寻找中点并检测环。
6. 链表相交、成环判断关注节点身份，而不是节点值。

第 7 讲：二叉树与递归遍历

0. 本讲的核心问题

链表中的节点最多只有一个后继；二叉树中的节点最多有两个子节点：左子树和右子树。

```
    1
   / \
  2   3
 / \
4   5
```

树题的关键不是记住很多模板，而是建立一种递归视角：

> 如果左右子树的问题已经解决，当前节点怎样利用它们的结果得到整棵树的答案？

---

1. 基本术语

```
    1          ←     root
   / \
  2   3        ← 1
 / \
4   5          ←
```

- 根：最上面的节点；
- 父节点：直接连接到下方节点的节点；
- 子节点：某节点直接连接的下一层节点；
- 叶节点：没有子节点；
- 子树：以某节点为根的完整树；
- 深度：从根到该节点经过的边数或节点数，必须看题目定义；
- 高度：从该节点向下到最深叶子的距离；
- 层：处于相同深度的节点集合。

面试题对深度、高度有时采用不同起点定义。不要只背数字，应根据函数含义明确约定。

---

2. Python 中的二叉树节点

```
[python]
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

手动创建：

```
[python]
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
```

空树：

```
[python]
root = None
```

LeetCode 通常已经构造好树，并把根节点传给方法。

---

3. 为什么树天然适合递归

一棵非空二叉树由三部分组成：

```
+     +
```

左右子树本身仍然是二叉树，所以同一个函数可以继续处理它们。

递归函数必须说清：

1. 这个函数接收一棵什么树？
2. 它返回什么含义的结果？
3. 空树应该返回什么？
4. 当前节点怎样组合左右子树的结果？

不要从“代码第一行写什么”开始。先写函数契约。

---

4. 深度优先遍历 DFS

DFS 会沿一条分支深入到底，再返回处理其他分支。

三种经典顺序根据“当前节点何时处理”命名。

4.1 前序：根、左、右

```
[python]
def preorder(root):
    result = []

    def dfs(node):
        if node is None:
            return

        result.append(node.val)
        dfs(node.left)
        dfs(node.right)

    dfs(root)
    return result
```

对于：

```
    1
   / \
  2   3
 / \
4   5
```

前序结果：

```
1, 2, 4, 5, 3
```

4.2 中序：左、根、右

```
[python]
def inorder(root):
    result = []

    def dfs(node):
        if node is None:
            return

        dfs(node.left)
        result.append(node.val)
        dfs(node.right)

    dfs(root)
    return result
```

结果：

```
4, 2, 5, 1, 3
```

二叉搜索树的中序遍历会得到有序序列。

4.3 后序：左、右、根

```
[python]
def postorder(root):
    result = []

    def dfs(node):
        if node is None:
            return

        dfs(node.left)
        dfs(node.right)
        result.append(node.val)

    dfs(root)
    return result
```

结果：

```
4, 5, 2, 3, 1
```

后序适合“先得到子树结果，再计算当前节点”的问题。

---

5. 两种常见递归写法

5.1 使用外部状态收集结果

遍历题常见：

```
[python]
result = []

def dfs(node):
    if node is None:
        return
    result.append(node.val)
    dfs(node.left)
    dfs(node.right)
```

函数主要产生副作用，不依赖返回值组合。

5.2 每次递归返回一个明确结果

求高度时更自然：

```
[python]
def max_depth(root):
    if root is None:
        return 0

    left_depth = max_depth(root.left)
    right_depth = max_depth(root.right)

    return 1 + max(left_depth, right_depth)
```

函数契约：

> `max_depth(node)` 返回以 `node` 为根的树的最大节点层数。

空树深度为 0。非空树的最大深度等于较深子树的深度加 1。

这种写法像数学递推：

\[D(T)= \begin{cases} 0, & T=\varnothing\\ 1+\max(D(T_L),D(T_R)), & \text{otherwise} \end{cases} \]

---

6. 怎样选择前序还是后序

可以问：

> 当前节点的工作需要先知道子树答案吗？

- 不需要，先处理当前节点再向下：偏前序；
- 需要左右子树结果才能计算当前答案：偏后序；
- 在二叉搜索树中按顺序访问：中序。

这不是绝对规则，但能帮助设计。

---

7. 翻转二叉树

目标是交换每个节点的左右子树：

```
[python]
def invert_tree(root):
    if root is None:
        return None

    root.left, root.right = root.right, root.left
    invert_tree(root.left)
    invert_tree(root.right)

    return root
```

这里先交换再递归是前序思路。也可以先分别翻转原左右子树，再交换结果：

```
[python]
def invert_tree(root):
    if root is None:
        return None

    left_inverted = invert_tree(root.left)
```

```
        right_inverted = invert_tree(root.right)
        root.left = right_inverted
        root.right = left_inverted

        return root
```

两个版本都正确，但不要在交换后又误把旧的子树方向当成未交换状态。

---

8. 判断两棵树是否相同

函数契约：

> `is_same_tree(p, q)` 判断以 `p` 和 `q` 为根的两棵树是否结构相同且对应值相等。

```
[python]
def is_same_tree(p, q):
    if p is None and q is None:
        return True

    if p is None or q is None:
        return False

    if p.val != q.val:
        return False

    return (
        is_same_tree(p.left, q.left)
        and is_same_tree(p.right, q.right)
    )
```

边界分三种：

1. 两者都空：相同；
2. 只有一个空：不同；
3. 都非空：比较值和左右子树。

---

9. 对称二叉树

对称不是“左右子树相同”，而是镜像：

```
        ⇔
        ⇔
```

```
[python]
def is_symmetric(root):
    def is_mirror(left, right):
        if left is None and right is None:
            return True

        if left is None or right is None:
            return False

        if left.val != right.val:
            return False

        return (
            is_mirror(left.left, right.right)
            and is_mirror(left.right, right.left)
        )

    if root is None:
```

```
            return True

        return is_mirror(root.left, root.right)
```

关键在于给辅助函数定义正确的契约：它判断两棵树是否互为镜像。

---

10. 路径和：向下传递状态

问题：是否存在从根到叶子的路径，使节点和等于 `target_sum`。

可以把剩余目标向下传：

```
[python]
def has_path_sum(root, target_sum):
    if root is None:
        return False

    remaining = target_sum - root.val

    if root.left is None and root.right is None:
        return remaining == 0

    return (
        has_path_sum(root.left, remaining)
        or has_path_sum(root.right, remaining)
    )
```

为什么必须检查叶节点？

题目要求根到叶路径。如果在中间节点就令 `remaining == 0` 并返回真，可能把未到叶子的前缀误认为完整路径。

这种模式是：

> 从父节点向子节点传递“到当前为止的状态”。

---

11. 直径：同时返回局部信息并更新全局答案

二叉树直径是任意两个节点间最长路径的边数。最长路径不一定经过根。

对每个节点，经过它的最长路径为：

\[left_height+right_height \]

递归返回当前子树高度，同时更新全局最大直径：

```
[python]
def diameter_of_binary_tree(root):
    best = 0

    def height(node):
        nonlocal best

        if node is None:
            return 0

        left_height = height(node.left)
        right_height = height(node.right)

        best = max(best, left_height + right_height)

        return 1 + max(left_height, right_height)

    height(root)
```

```
        return best
```

这里要区分：

- `height` 返回给父节点的是一条向下的最长路径；
- `best` 保存任意节点处左右路径连接得到的全局最大值。

这是树形递归中很重要的设计模式。

---

12. 迭代 DFS：显式使用栈

递归使用系统调用栈。也可以自己维护栈：

```
[python]
def preorder_iterative(root):
    if root is None:
        return []

    result = []
    stack = [root]

    while stack:
        node = stack.pop()
        result.append(node.val)

        if node.right is not None:
            stack.append(node.right)
        if node.left is not None:
            stack.append(node.left)

    return result
```

为什么先压右节点?

栈后进先出。为了先处理左节点，需要让左节点后入栈。

中序和后序的迭代写法更复杂，初学阶段优先掌握递归版。

---

13. 复杂度与树高

访问所有节点的 DFS：

```
O(n)
```

递归额外空间取决于树高 `h`：

```
O(h)
```

平衡树：

```
h = O(log n)
```

完全倾斜的树：

```
1
 \
  2
   \
    3
```

此时：

```
h = O(n)
```

所以递归空间最坏为 `O(n)`。Python 中极深的树还可能触发递归深度限制。

14. 常见错误

14.1 没有定义递归函数的返回含义

写到一半不知道应该 `return node`、数字还是布尔值。应先写函数契约。

14.2 空树返回值不符合递推

- 求深度：空树返回 0；
- 判断相同：两者都空返回真；
- 路径和：空树通常返回假。

空树返回什么由函数含义决定，不是统一模板。

14.3 忘记使用递归返回值

```
[python]
max_depth(root.left)
max_depth(root.right)
```

如果不保存结果，就无法组合得到当前答案。

14.4 混淆节点数和边数

最大深度与直径可能采用不同单位。阅读题目并保持一致。

14.5 把根到节点路径误认为根到叶路径

叶节点条件是左右孩子都为空。

14.6 共享可变默认参数

不要写：

不要把函数定义写成 `def dfs(node, result=[]):`。

默认列表会在多次调用间共享。应在外层创建结果列表。

15. 本讲检查表

```
1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
```

16. 本讲总结

1. 二叉树由根、左子树和右子树递归组成。
2. 前序、中序、后序的区别是当前节点的处理位置。
3. 写递归前先定义函数契约和空树返回值。

4. 最大深度是典型的“组合左右子树返回值”。
5. 路径和是“向下传递状态”，直径是“返回局部信息并更新全局答案”。
6. DFS 时间通常为 `O(n)`，递归空间为 `O(h)`。

第 8 讲：二叉搜索树、层序遍历与树的综合题

0. 本讲的核心问题

普通二叉树只规定每个节点最多有两个孩子，并不规定值的大小关系。

二叉搜索树 BST 进一步要求：

```
        <     <
```

这种全局有序性使查找可以像二分查找一样排除一整棵子树。

本讲还会学习树的 BFS：层序遍历。DFS 关心递归结构，BFS 关心同一深度的节点。

---

1. BST 的定义是全局约束

正确示例：

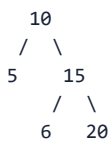


对于每个节点：

- 左子树所有值都比它小；
- 右子树所有值都比它大；
- 左右子树本身也必须是 BST。

注意“所有值”，不只是直接孩子。

下面不是 BST：



虽然 `6 < 15`，但 `6` 位于根 `10` 的右子树，却小于 `10`，违反全局约束。

重复值如何处理取决于题目定义。LeetCode 的验证 BST 通常要求严格不等，不允许重复值。

---

2. BST 搜索

```
[python]
def search_bst(root, target):
    current = root

    while current is not None:
        if target == current.val:
            return current

        if target < current.val:
            current = current.left
        else:
            current = current.right

    return None
```

每次比较后都排除一整棵子树。

复杂度取决于树高 `h`：

```
O(h)
O(1)  #
```

平衡树中 `h=O(log n)`；倾斜树中 `h=O(n)`。BST 不自动保证平衡。

递归版本：

```
[python]
def search_bst(root, target):
    if root is None or root.val == target:
        return root

    if target < root.val:
        return search_bst(root.left, target)

    return search_bst(root.right, target)
```

---

3. BST 插入

沿搜索路径找到空位置：

```
[python]
def insert_into_bst(root, value):
    if root is None:
        return TreeNode(value)

    if value < root.val:
        root.left = insert_into_bst(root.left, value)
    elif value > root.val:
        root.right = insert_into_bst(root.right, value)

    return root
```

这个版本忽略重复值。具体策略必须服从题目定义。

为什么需要：

```
[python]
root.left = insert_into_bst(root.left, value)
```

因为当左子树为空时，递归会创建并返回新节点，父节点必须把 `.left` 连接到它。

---

4. 中序遍历为什么有序

中序顺序是：

```
          →     →
```

BST 中：

```
             <     <
```

因此中序遍历得到严格递增序列。

```
[python]
def inorder_values(root):
    result = []

    def dfs(node):
        if node is None:
            return

        dfs(node.left)
        result.append(node.val)
        dfs(node.right)

    dfs(root)
    return result
```

这条性质可以解决验证 BST、第 K 小元素、寻找相邻差值等问题。

---

5. 验证 BST：错误方法与正确方法

5.1 只比较直接孩子是不够的

错误思路：

```
[python]
def only_checks_children(node):
    if node.left and node.left.val >= node.val:
        return False
    if node.right and node.right.val <= node.val:
        return False
    return True
```

它无法发现右子树深处出现一个小于祖先的值。

5.2 方法一：为每个节点维护允许区间

根允许范围是：

```
(-∞, +∞)
```

进入左子树时，上界变成当前值；进入右子树时，下界变成当前值。

```
[python]
def is_valid_bst(root):
    def validate(node, lower, upper):
        if node is None:
            return True

        if not (lower < node.val < upper):
            return False

        return (
            validate(node.left, lower, node.val)
            and validate(node.right, node.val, upper)
        )
```

```
        return validate(root, float("-inf"), float("inf"))
```

函数契约：

> `validate(node, lower, upper)` 判断整棵子树的所有值是否严格位于给定开区间。

5.3 方法二：检查中序序列严格递增

不必保存完整列表，只保存前一个访问值：

```
[python]
def is_valid_bst(root):
    previous = None

    def inorder(node):
        nonlocal previous

        if node is None:
            return True

        if not inorder(node.left):
            return False

        if previous is not None and node.val <= previous:
            return False

        previous = node.val

        return inorder(node.right)

    return inorder(root)
```

不要用某个普通数字作为初始 `previous`，否则可能与合法节点值冲突。使用 `None` 或负无穷。

---

6. 第 K 小元素

BST 中序序列递增，所以中序访问的第 `k` 个节点就是第 K 小。

```
[python]
def kth_smallest(root, k):
    stack = []
    current = root

    while stack or current is not None:
        while current is not None:
            stack.append(current)
            current = current.left

        current = stack.pop()
        k -= 1

        if k == 0:
            return current.val

        current = current.right
```

这个迭代中序可以在找到第 K 个元素时提前停止，不必遍历整棵树。

栈中保存“已经到达但尚未访问值的祖先节点”。

最坏时间 `O(n)`，额外空间 `O(h)`；如果 `k` 很小，可能提前结束。

---

7. BST 最近公共祖先

给定 BST 中两个节点 `p q`：

- 如果二者都小于当前值，公共祖先在左边；
- 如果二者都大于当前值，公共祖先在右边；
- 否则当前节点位于二者之间，或者等于其中之一，就是最近公共祖先。

```
[python]
def lowest_common_ancestor_bst(root, p, q):
    current = root

    while current is not None:
        if p.val < current.val and q.val < current.val:
            current = current.left
        elif p.val > current.val and q.val > current.val:
            current = current.right
        else:
            return current

    return None
```

时间 `O(h)`，额外空间 `O(1)`。

普通二叉树没有大小关系，需要不同方法。

---

8. 普通二叉树的最近公共祖先

函数契约：

> 返回当前子树中 `p q` 的最近公共祖先；如果只找到其中一个，则把找到的节点向上传递；如果都没找到，返回 `None`。

```
[python]
def lowest_common_ancestor(root, p, q):
    if root is None or root is p or root is q:
        return root

    left_result = lowest_common_ancestor(root.left, p, q)
    right_result = lowest_common_ancestor(root.right, p, q)

    if left_result is not None and right_result is not None:
        return root

    if left_result is not None:
        return left_result

    return right_result
```

如果左右子树各返回一个非空结果，说明 `p q` 分处两侧，当前节点就是最近公共祖先。

这道题再次体现：递归代码是否自然，取决于返回值含义是否定义清楚。

---

9. 层序遍历：树上的 BFS

目标：按层返回节点值。

```
    3
   / \
  9   20
```

```
    /  \
   15   7
```

输出：

```
[python]
[[3], [9, 20], [15, 7]]
```

使用队列：

```
[python]
from collections import deque

def level_order(root):
    if root is None:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level = []

        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)

            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)

        result.append(level)

    return result
```

循环开始时，队列中的前 `level_size` 个节点恰好属于当前层。处理它们时加入的是下一层。

时间 `O(n)`，空间最坏 `O(w)`，其中 `w` 是树的最大宽度，最坏可达 `O(n)`。

---

10. 层序遍历的变形

10.1 右视图

从右侧看，每层只能看到最右节点：

```
[python]
from collections import deque

def right_side_view(root):
    if root is None:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)

        for index in range(level_size):
            node = queue.popleft()

            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)
```

```
                if index == level_size - 1:
                    result.append(node.val)

        return result
```

10.2 每层平均值

对本层累加并除以 `level_size`。

10.3 锯齿形层序遍历

仍然按正常 BFS 收集每层，只在输出时根据层号决定是否反转该层。不要为了输出方向破坏队列的正常扩展逻辑。

---

11. 判断完全二叉树的直觉

完全二叉树要求除最后一层外都填满，最后一层从左到右连续填充。

层序遍历中，一旦遇到空位置，之后不应再出现非空节点。

```
[python]
from collections import deque

def is_complete_tree(root):
    queue = deque([root])
    seen_none = False

    while queue:
        node = queue.popleft()

        if node is None:
            seen_none = True
            continue

        if seen_none:
            return False

        queue.append(node.left)
        queue.append(node.right)

    return True
```

这里故意把 `None` 也入队，用它表示树形结构中的空位置。

---

12. BST 删除的三种情况

删除节点比插入复杂。设要删除的节点为 `node`：

1. 没有子节点：直接删除；
2. 只有一个子节点：用唯一子节点替代它；
3. 有两个子节点：用右子树最小节点（中序后继）或左子树最大节点替代，再删除那个替代节点。

递归实现：

```
[python]
def delete_node(root, key):
    if root is None:
        return None

    if key < root.val:
        root.left = delete_node(root.left, key)
    elif key > root.val:
        root.right = delete_node(root.right, key)
```

```
        else:
            if root.left is None:
                return root.right
            if root.right is None:
                return root.left

            successor = root.right
            while successor.left is not None:
                successor = successor.left

            root.val = successor.val
            root.right = delete_node(root.right, successor.val)

        return root
```

这部分理解即可，初学阶段优先熟练搜索、验证和中序性质。

---

13. 怎样选择 DFS、BFS 或 BST 性质

问题	优先思路
最大深度、直径、路径	DFS 递归
按层输出、最小层数	BFS 队列
BST 查找或插入	比较大小，只走一侧
BST 第 K 小	中序遍历
BST 验证	上下界或中序递增
BST 最近公共祖先	利用大小关系
普通树最近公共祖先	后序递归

同一道题可能有多种方法。选择时看题目真正需要的是层、路径、子树结果还是有序性。

---

14. 复杂度中的平衡与倾斜

BST 搜索常被简单说成 `O(log n)`，但只有树较平衡时才成立。

平衡：

```
    4
   / \
  2   6
```

高度约为 `log n`。

倾斜：

```
1
 \
  2
   \
    3
```

高度为 `n`，搜索退化到 `O(n)`。

因此更准确的写法是：

```
O(h)
O(log n)
O(n)
```

15. 常见错误

15.1 把 BST 条件只检查到直接孩子

必须维护来自所有祖先的范围，或验证完整中序序列。

15.2 中序遍历的 `previous` 没有正确更新

比较当前节点后，应立即更新再进入右子树。

15.3 层序遍历动态使用 `len(queue)`

处理一层前保存 `level_size`，否则下一层节点会混入当前层。

15.4 忘记空树

层序遍历若直接 `deque([None])`，随后访问 `.val` 会出错，除非算法特意处理 `None`。

15.5 默认 BST 一定平衡

复杂度应先写成 `O(h)`。

15.6 普通树 LCA 和 BST LCA 混用

只有 BST 才能利用值的大小决定方向。

16. 本讲检查表

```
1.          BST
2.
3.
4.
5.
6.
7. BFS           level_size
8.
9.          O(h)    O(n)
10.
```

17. 本讲总结

1. BST 的约束作用于整个左右子树，不只是直接孩子。
2. BST 搜索、插入和删除复杂度是 `O(h)`，不一定总是 `O(log n)`。
3. 中序遍历 BST 得到严格递增序列。
4. 验证 BST 可使用祖先上下界或中序递增性质。
5. 第 K 小元素是中序遍历的第 K 个节点。
6. 层序遍历使用队列，并在每层开始前固定层大小。
7. 普通树和 BST 的最近公共祖先使用不同信息。

第 9 讲：堆与优先队列——反复取出当前最重要的元素

0. 本讲的核心问题

假设数据不断到来，而你需要反复回答：

- 当前最小的任务是什么？
- 当前最大的 `k` 个数是什么？
- 第 `k` 大的元素是什么？
- 多个候选状态中，下一步应优先处理哪一个？

每次都把全部数据重新排序当然可以，但通常太浪费。

堆解决的问题是：

> 不要求所有元素完全有序，只保证最重要的元素随时位于顶部。

Python 的 `heapq` 默认实现最小堆，因此堆顶 `heap[0]` 始终是当前最小元素。

---

1. 优先队列与普通队列的区别

普通队列按照进入顺序处理：

优先队列按照优先级处理：

例如医院分诊不会单纯按到达时间处理，而会优先处理情况更紧急的患者。

优先队列是一种抽象接口：

```
push(item)
peek()
pop()
```

堆是实现优先队列的常用数据结构。

---

2. 堆的结构

最小堆满足：

> 每个父节点的值都不大于它的子节点。

例如：



```
          9   8  7
```

父节点不一定比整棵子树中的所有元素都先经过排序比较，但由于堆性质沿路径成立，根节点一定是全局最小值。

注意：

```
[2, 5, 4, 9, 8, 7]
```

是一个合法最小堆，但它不是一个完全排序的数组。`5` 可以出现在 `4` 前面。

2.1 为什么堆可以存在 Python list 中

堆是一棵完全二叉树：除了最后一层以外每层都填满，最后一层从左向右填充。

因此不需要为每个节点保存 `left` 和 `right` 引用，可以按层放入数组：

```
 0  1  2  3  4  5
[2, 5, 4, 9, 8, 7]
```

对下标 `i`：

```
[python]
left = 2 * i + 1
right = 2 * i + 2
parent = (i - 1) // 2
```

在 LeetCode 中通常不需要自己实现上浮和下沉，但理解数组与树的对应关系有助于理解复杂度。

---

3. 插入和删除为什么是 `O(log n)`

3.1 插入：先放到末尾，再向上调整

把 `1` 加入下面的堆：

```
[2, 5, 4, 9, 8, 7]
[2, 5, 4, 9, 8, 7, 1]
```

新元素可能小于父节点，因此不断与父节点交换，直到恢复堆性质。这叫上浮。

完全二叉树的高度是 `O(log n)`，所以最多交换 `O(log n)` 次。

3.2 删除堆顶：末尾补到根，再向下调整

删除根节点后，为了保持完全二叉树形状：

1. 把最后一个元素移到根；
2. 比较它与子节点；
3. 与更小的子节点交换；
4. 继续向下，直到恢复堆性质。

这叫下沉，同样最多经过树高 `O(log n)`。

3.3 常见复杂度

操作	时间复杂度
查看堆顶 `heap[0]`	`O(1)`
插入 `heappush`	`O(log n)`
弹出堆顶 `heappop`	`O(log n)`
把列表原地建堆 `heapify`	`O(n)`
在堆中查找任意值	`O(n)`

堆只对“最优元素”提供快速访问，并不擅长查找任意元素。

---

4. Python `heapq` 的基本使用

```
[python]
import heapq

heap = []

heapq.heappush(heap, 5)
heapq.heappush(heap, 2)
heapq.heappush(heap, 8)

smallest = heap[0]          # 2
x = heapq.heappop(heap)     # 2
```

不要通过打印结果判断堆是否正确：

```
[python]
print(heap)
```

输出只保证满足堆性质，不保证从小到大排列。

如果需要把所有元素从小到大取出：

```
[python]
result = []

while heap:
    result.append(heapq.heappop(heap))
```

4.1 对已有列表建堆

```
[python]
nums = [7, 2, 9, 1, 5]
heapq.heapify(nums)
```

`heapify` 会原地改变 `nums`，时间为 `O(n)`。

如果不能修改原列表：

```
[python]
heap = nums.copy()
heapq.heapify(heap)
```

---

5. Python 怎样实现最大堆

`heapq` 传统接口是最小堆。为了优先取出最大值，可以存相反数：

```
[python]
```

```
import heapq

max_heap = []

for x in [4, 9, 2]:
    heapq.heappush(max_heap, -x)

largest = -heapq.heappop(max_heap)  # 9
```

真实数值越大，相反数越小，因此会先从最小堆弹出。

常见错误是只在压入时取负，弹出后忘记恢复：

```
[python]
x = -heapq.heappop(max_heap)
```

变量名最好直接写成 `max_heap`，提醒自己其中存的是相反数。

---

6. 堆中存元组：优先级与数据

实际题目往往不只保存一个数字，而要同时保存优先级和状态：

```
[python]
heapq.heappush(heap, (priority, item))
```

Python 比较元组时先比较第一个元素；相等时再比较第二个：

```
(2, "B") < (5, "A")
```

因此可以写：

```
[python]
tasks = []
heapq.heappush(tasks, (3, "   "))
heapq.heappush(tasks, (1, "    "))

priority, task = heapq.heappop(tasks)
```

6.1 优先级相同时的比较问题

如果第二项是不可比较的自定义对象，优先级相同时可能报错。可以加入唯一序号：

```
[python]
heapq.heappush(heap, (priority, sequence, item))
```

`sequence` 可以使用递增整数，保证前两项已经能确定顺序，不需要比较 `item`。

---

7. 核心模式：维护大小为 `k` 的堆

题目：在大量数字中找最大的 `k` 个。

不必保存全部数字的排序结果。只维护当前最大的 `k` 个：

```
[python]
import heapq

def top_k_largest(nums, k):
    heap = []

    for x in nums:
        heapq.heappush(heap, x)

        if len(heap) > k:
            heapq.heappop(heap)

    return heap
```

这里为什么使用最小堆？

> 堆中保存的是当前最大的 k 个数；堆顶是其中最小的一个，也就是最应该被淘汰的门槛。

处理每个数后：

```
heap =          k        k
```

这就是循环不变量。

7.1 复杂度

堆的大小最多为 k：

```
O(n log k)
O(k)
```

如果完整排序，则时间通常为 O(n log n)。

当 k 远小于 n 时，小堆更合适。

---

8. 第 k 大元素

维护最大的 k 个元素后，堆顶正好是它们中最小的，即整个数组第 k 大：

```
[python]
import heapq

def find_kth_largest(nums, k):
    heap = []

    for x in nums:
        heapq.heappush(heap, x)

        if len(heap) > k:
            heapq.heappop(heap)

    return heap[0]
```

也可以使用：

```
[python]
heapq.nlargest(k, nums)
```

但学习阶段应理解“大小为 k 的最小堆”为什么成立。

8.1 k 大与 k 小怎样选择堆

容易混淆时，不要死记口诀，问：

> 堆满以后，下一次最需要快速淘汰谁？

- 保留最大的 k 个：淘汰当前最小者，所以用最小堆；
- 保留最小的 k 个：淘汰当前最大者，所以用最大堆。

---

9. 高频题：前 k 个高频元素

先使用哈希表统计频率：

```
[python]
counts = {}

for x in nums:
    counts[x] = counts.get(x, 0) + 1
```

再维护频率最大的 `k` 项：

```
[python]
import heapq

def top_k_frequent(nums, k):
    counts = {}

    for x in nums:
        counts[x] = counts.get(x, 0) + 1

    heap = []

    for value, frequency in counts.items():
        heapq.heappush(heap, (frequency, value))

        if len(heap) > k:
            heapq.heappop(heap)

    return [value for frequency, value in heap]
```

如果有 `m` 个不同元素：

```
 O(n)
O(m log k)
 O(m + k)
```

返回顺序通常不重要；如果题目要求特定顺序，需要额外处理。

---

10. 高频题：合并多个有序链表

有 `k` 条有序链表。任何时刻，答案中的下一个最小节点一定来自各链表当前头节点中的最小者。

因此堆中不需要放入所有节点，只放每条链表当前可选的第一个节点。

```
[python]
import heapq

def merge_k_lists(lists):
    heap = []

    for i, node in enumerate(lists):
        if node is not None:
            heapq.heappush(heap, (node.val, i, node))

    dummy = ListNode()
    tail = dummy

    while heap:
        value, list_index, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next

        if node.next is not None:
            heapq.heappush(
                heap,
                (node.next.val, list_index, node.next),
            )

    return dummy.next
```

为什么保存 `list_index`？

当两个节点值相等时，Python 会继续比较元组第二项。整数下标可以稳定地区分它们，避免直接比较 `ListNode` 对象。

若总节点数为 `n`，堆大小最多为 `k`：

```
O(n log k)
O(k)   #
```

---

11. 堆与排序的选择

更适合排序

- 最终需要所有元素完全有序；
- 数据量不大；
- 只进行一次离线处理；
- 排序后的顺序还会被多次使用。

更适合堆

- 只需要最大或最小的少量元素；
- 数据持续到来；
- 要反复取出当前最优元素；
- `k` 远小于 `n`；
- 搜索过程中候选状态的优先级不断变化。

不要因为题目出现“最大”或“最小”就机械使用堆。只求一次最大值时，线性扫描 `O(n)` 更简单。

---

12. 常见错误

12.1 把堆当作有序数组

只能保证 `heap[0]` 是最小值，不能假设 `heap[1]` 是第二小。

12.2 直接使用 `heap.pop(0)`

这样既破坏堆操作接口，又会触发列表整体移动。应使用：

```
[python]
heapq.heappop(heap)
```

12.3 最大堆忘记取负两次

压入负数，弹出后也要恢复符号。

12.4 `k` 的含义混乱

先明确堆中保存的是“要留下的人”，还是“等待处理的候选”。写出一句不变量再编码。

12.5 元组中的对象不可比较

加入唯一序号：

```
[python]
(priority, sequence, object)
```

12.6 在空堆上读取 `heap[0]`

先确认：

```
[python]
if heap:
    smallest = heap[0]
```

12.7 修改了不该修改的输入

`heapify(nums)` 会原地改变 `nums`。题目不允许修改时先复制。

---

13. 本讲检查表

```
1.
2.
3.
4.                k
5.
6.
7.
8.
9.          heapify
10.            n    k
```

14. 本讲总结

1. 堆只维护局部顺序，堆顶是当前最优元素。
2. Python `heapq` 默认是最小堆；最大堆常用相反数模拟。
3. 插入和弹出堆顶为 `O(log n)`，查看堆顶为 `O(1)`。
4. Top K 常用大小为 `k` 的堆，把复杂度降为 `O(n log k)`。
5. 堆中可以保存 `(  ,  ,  )`。
6. 解题前先写清楚：堆里保留什么，堆顶为什么是下一步需要的元素。

第 10 讲：图、DFS 与 BFS——在关系网络中搜索

0. 本讲的核心问题

树中的每个节点通常只有一个父节点；图则允许更一般的关系：

- 城市之间有道路；
- 用户之间有关注关系；
- 课程之间有先修关系；
- 网格中的格子彼此相邻；
- 一个状态可以通过若干操作转移到其他状态。

图算法的共同问题是：

> 从一个状态出发，怎样系统地找到能够到达的其他状态，并避免无限重复？

本讲重点不是记大量图论名词，而是建立统一模型：

---

1. 图的基本概念

图由节点和边组成，常记为：

```
G = (V, E)
```

- `V`：节点集合；
- `E`：边集合。

1.1 无向图

边没有方向：

```
A — B
```

表示 A 能到 B，B 也能到 A。道路双向通行时可以建模为无向图。

1.2 有向图

边有方向：

```
A → B
```

表示可以从 A 到 B，不一定能从 B 到 A。课程先修关系和网页链接常是有向图。

1.3 加权图与无权图

无权图只关心是否有边，或把每条边都看作代价 1。

加权图的边带有距离、费用、时间等代价：

```
A --5--> B
```

普通 BFS 只直接保证无权图或等权图的最短边数。边权不同的最短路需要其他算法，例如 Dijkstra。

1.4 路径、环与连通分量

- 路径：沿边经过的一系列节点；
- 环：从某节点出发，沿边最终回到它；
- 连通分量：无向图中彼此可以到达的一组节点。

---

2. 图怎样存入代码

2.1 邻接表

```python
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"],
}
```

`graph[node]` 保存从 `node` 能直接到达的邻居。

邻接表适合大多数 LeetCode 图题。如果有 `V` 个节点、`E` 条边：

```
O(V + E)
```

无向图的一条边通常在邻接表中出现两次：

```python
graph[a].append(b)
graph[b].append(a)
```

2.2 邻接矩阵

用二维数组记录任意两点之间是否有边：

```python
matrix[i][j] = 1
```

访问一条特定边是 `O(1)`，但无论边多不多都需要 `O(V²)` 空间。节点很多、边很少时通常不如邻接表。

2.3 边列表

```python
edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
]
```

题目经常以边列表给出输入，需要先转成邻接表。

```python
def build_undirected_graph(n, edges):
    graph = [[] for _ in range(n)]

    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
```

```
    return graph
```

2.4 网格也是图

网格中的可通行格子是节点；上下左右的合法移动是边。

```
[python]
directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
```

很多看似没有“图”的题，本质上仍是在隐式图中搜索。

---

3. DFS：沿一条路尽量深入

深度优先搜索 DFS 的策略是：

> 选择一个尚未访问的邻居，沿它继续深入；走不下去时再返回。

可以用递归，也可以用显式栈。

3.1 递归 DFS 模板

```
[python]
def dfs(graph, node, visited):
    visited.add(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

调用：

```
[python]
visited = set()
dfs(graph, start, visited)
```

递归函数的含义应该能说成一句话：

> `dfs(node)` 访问 `node` 能够到达、且此前未访问的全部节点。

3.2 为什么先标记再递归

无向图中 A 的邻居有 B，B 的邻居又有 A：

```
A — B
```

如果进入 A 后不立即标记，A 调用 B，B 又调用 A，会无限递归。

因此通常遵守：

```
→      →
```

3.3 显式栈 DFS

```
[python]
def dfs_iterative(graph, start):
    stack = [start]
    visited = {start}

    while stack:
        node = stack.pop()

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                stack.append(neighbor)

    return visited
```

这与递归 DFS 使用相同思想，只是把系统调用栈换成自己维护的 `stack`。

---

4. BFS：按距离一层层扩展

BFS 使用队列：

```
[python]
from collections import deque

def bfs(graph, start):
    queue = deque([start])
    visited = {start}

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return visited
```

BFS 与第 5 讲相同：入队时标记，避免同一个节点被多个前驱重复加入队列。

4.1 DFS 与 BFS 如何选择

目标	更自然的方法
判断是否可达	DFS 或 BFS
遍历整个连通块	DFS 或 BFS
无权图最少边数	BFS
枚举路径、回溯选择	DFS
层序、同时扩散	BFS
递归描述子问题	DFS

很多题两者都能完成。选择你更容易维护状态和证明正确性的写法。

---

5. 图搜索的统一模板

无论 DFS 还是 BFS，都要回答四个问题：

1.
2.
3.
4.

抽象模板：

```
worklist = [start]
visited = {start}

while worklist:
    state = remove_one(worklist)

    if is_goal(state):
        return answer

    for next_state in neighbors(state):
        if next_state not in visited:
            visited.add(next_state)
            add(worklist, next_state)
```

- `worklist` 是栈：DFS；
- `worklist` 是队列：BFS；
- `worklist` 是堆：按优先级搜索。

理解这点后，很多算法不再是互不相关的模板。

---

6. 核心题：岛屿数量

网格中：

- `"1"` 表示陆地；
- `"0"` 表示水；
- 上下左右相邻陆地属于同一座岛。

思路：

1. 扫描每个格子；
2. 如果遇到尚未访问的陆地，发现一座新岛；
3. 从这里 DFS，把整座岛全部标记；
4. 继续扫描。

```
[python]
def num_islands(grid):
    if not grid or not grid[0]:
        return 0

    rows = len(grid)
    cols = len(grid[0])
    visited = set()
    islands = 0
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    def dfs(row, col):
        visited.add((row, col))
```

```
            for dr, dc in directions:
                nr = row + dr
                nc = col + dc

                if not (0 <= nr < rows and 0 <= nc < cols):
                    continue
                if grid[nr][nc] != "1":
                    continue
                if (nr, nc) in visited:
                    continue

                dfs(nr, nc)

        for row in range(rows):
            for col in range(cols):
                if grid[row][col] == "1" and (row, col) not in visited:
                    islands += 1
                    dfs(row, col)

        return islands
```

6.1 为什么岛屿数在启动 DFS 时增加

每次从未访问陆地启动 DFS，说明此前的所有岛屿搜索都没有到达它，因此它属于一个新连通分量。

DFS 内部访问到的其他陆地与起点连通，不应重复计数。

6.2 使用网格本身标记

如果允许修改输入，可以把访问过的陆地改为水：

```
[python]
grid[row][col] = "0"
```

这样不需要额外 `visited`，但会改变输入。

复杂度：

```
O(rows × cols)
O(rows × cols)   # visited
```

---

7. 连通分量数量

给定 `n` 个节点和无向边，求有多少个彼此独立的连通分量。

```
[python]
def count_components(n, edges):
    graph = [[] for _ in range(n)]

    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    visited = set()
    components = 0

    for start in range(n):
        if start in visited:
            continue

        components += 1
        stack = [start]
        visited.add(start)

        while stack:
            node = stack.pop()
```

```
                for neighbor in graph[node]:
                    if neighbor not in visited:
                        visited.add(neighbor)
                        stack.append(neighbor)

        return components
```

这个模式与岛屿数量完全相同：

```
                →       +1 →
```

---

8. 图中的环

8.1 无向图判断环

在无向图 DFS 中，从 A 走到 B 后，B 一定能看到 A。但这条“回到父节点”的边不代表环。

因此递归时保存父节点：

```
[python]
def has_cycle_undirected(graph):
    visited = set()

    def dfs(node, parent):
        visited.add(node)

        for neighbor in graph[node]:
            if neighbor == parent:
                continue

            if neighbor in visited:
                return True

            if dfs(neighbor, node):
                return True

        return False

    for node in graph:
        if node not in visited and dfs(node, None):
            return True

    return False
```

8.2 有向图为什么更复杂

有向图中，指向“曾经访问过的节点”不一定形成环。只有指向当前递归路径上的节点，才说明出现回边。

可以使用三种状态：

```
0
1
2
```

```
[python]
def has_cycle_directed(graph):
    state = {node: 0 for node in graph}

    def dfs(node):
        if state[node] == 1:
            return True
        if state[node] == 2:
            return False

        state[node] = 1
```

```
                for neighbor in graph[node]:
                    if dfs(neighbor):
                        return True

                state[node] = 2
                return False

            for node in graph:
                if state[node] == 0 and dfs(node):
                    return True

            return False
```

`state == 1` 可以理解为递归栈中的节点。

---

9. 拓扑排序：处理先修关系

课程依赖：

```
A     B A → B
```

如果依赖中有环：

```
A → B → C → A
```

就无法完成所有课程。

拓扑排序为有向无环图产生一个满足所有先后关系的顺序。

9.1 入度

节点的入度是指向它的边数。

入度为 0 表示当前没有未完成的先修要求，可以立即处理。

9.2 Kahn 算法：基于 BFS

```
[python]
from collections import deque

def can_finish(num_courses, prerequisites):
    graph = [[] for _ in range(num_courses)]
    indegree = [0] * num_courses

    for course, prerequisite in prerequisites:
        graph[prerequisite].append(course)
        indegree[course] += 1

    queue = deque()

    for course in range(num_courses):
        if indegree[course] == 0:
            queue.append(course)

    completed = 0

    while queue:
        course = queue.popleft()
        completed += 1

        for next_course in graph[course]:
            indegree[next_course] -= 1

            if indegree[next_course] == 0:
                queue.append(next_course)

    return completed == num_courses
```

不变量：

> 队列中都是当前所有先修要求已经满足的课程。

如果最后处理数量小于课程总数，剩余节点被环互相依赖，没有任何一个能达到入度 0。

时间和空间均为 `O(V + E)`。

---

10. 克隆图：旧节点到新节点的映射

克隆图不能只复制节点值，还要复制连接关系，并保证每个旧节点只对应一个新节点。

使用字典：

```
      →

[python]
def clone_graph(node):
    if node is None:
        return None

    copies = {}

    def dfs(original):
        if original in copies:
            return copies[original]

        copy = Node(original.val)
        copies[original] = copy

        for neighbor in original.neighbors:
            copy.neighbors.append(dfs(neighbor))
```

```
            return copy

        return dfs(node)
```

为什么创建副本后要立即写入 `copies`，再递归邻居？

因为图可能有环。如果等邻居全部完成后再记录，沿环递归时会再次复制同一节点并无限递归。

`copies` 同时承担：

- visited；
- 旧节点到新节点的映射。

---

11. 递归深度与迭代写法

Python 的递归深度有限。一条很长的链状图可能触发：

```
RecursionError
```

面试题通常允许递归解法，但当数据可能很深时，应考虑显式栈。

递归和迭代的核心状态对应关系：

递归	迭代
函数参数	栈元素
调用函数	压栈
函数返回	弹栈
局部变量	栈元素中保存的状态

如果只需访问节点，简单栈即可；如果需要模拟“进入节点”和“退出节点”两个阶段，栈元素还要保存阶段信息。

---

12. 图搜索复杂度

邻接表中：

- 每个节点最多标记一次；
- 每条有向边最多检查一次；
- 无向边在邻接表中出现两次，但仍是常数倍。

因此 DFS 和 BFS 通常都是：

```
O(V + E)
O(V)
```

网格有 `rows × cols` 个格子，每个格子最多四条边：

```
O(rows × cols)
O(rows × cols)
```

不要把嵌套循环自动判断成 `O(n²)`。图搜索中，内层邻居循环在全局加起来只检查所有边。

---

13. 常见错误

13.1 无向边只加入一个方向

无向边 `(a, b)` 需要同时加入 `a → b` 和 `b → a`。

13.2 忘记 visited

图可能存在环，没有访问标记会无限循环。

13.3 标记太晚

BFS 和迭代 DFS 通常在加入工作表时就标记，避免重复安排。

13.4 只从节点 0 搜索

图可能不连通。若题目要求处理整张图，需要外层遍历所有节点。

13.5 混淆有向与无向的环检测

无向图要排除父边；有向图要区分“当前路径中”和“已经处理完”。

13.6 网格先访问再检查边界

必须先确认坐标合法，再读取 `grid[nr][nc]`。

13.7 拓扑图方向建反

若 `[course, prerequisite]` 表示先学 `prerequisite` 才能学 `course`，边应为：

```
prerequisite → course
```

13.8 把节点值当作节点身份

图节点可能值相同。克隆、相交等问题需要使用节点对象本身作为键。

---

14. 本讲检查表

```
1.
2.
3.
4.
5.
6.    DFS    BFS
7. visited
8.
9.
10.
11.
12.
```

15. 本讲总结

1. 图是节点与边构成的通用关系模型。
2. 邻接表是 LeetCode 中最常用的图表示，空间为 `O(V+E)`。
3. DFS 沿路径深入，BFS 按距离分层；两者都必须防止重复访问。
4. 连通分量题使用“外层扫描 + 内层搜完整块”。

5. 有向图环检测需要区分未访问、当前路径中、已经完成三种状态。
6. 拓扑排序不断处理入度为 0 的节点，可判断课程依赖是否可完成。
7. 图搜索复杂度通常为 `O(V+E)`，网格搜索通常为 `O(rows×cols)`。

第 11 讲：双指针——用两个位置共同描述搜索范围

0. 本讲的核心问题

很多数组和字符串题的暴力方法会枚举两个位置：

```
[python]
for i in range(n):
    for j in range(i + 1, n):
        ...
```

这通常需要 $O(n^2)$ 时间。

双指针的核心是：

> 利用数据的顺序、单调性或处理进度，让两个位置有目的地移动，从而排除大量不可能的组合。

双指针不是一种数据结构，而是一类变量设计方法。常见形态包括：

1. 左右指针：从两端向中间移动；
2. 快慢指针：同向移动，速度或职责不同；
3. 读写指针：一个读取输入，一个维护输出位置；
4. 两序列指针：分别指向两个数组当前未处理位置。

1. 双指针能够成立的前提

关键不是“用了两个变量”，而是移动一个指针后，能安全排除一批答案。

例如有序数组：

```
[1, 3, 4, 7, 9]
```

左右两端和太小：

```
nums[left] + nums[right] < target
```

如果继续保留较小的 `nums[left]`，把右指针左移只会让和更小，不可能得到目标。因此可以安全地执行：

```
[python]
left += 1
```

这就是单调性提供的排除依据。

每次移动前都应该能回答：

> 为什么被跳过的那些情况不可能是答案？

2. 左右指针：验证回文串

一个字符串是回文，当且仅当两端字符相同，并且内部子串也是回文。

```
[python]
def is_palindrome(text):
    left = 0
    right = len(text) - 1

    while left < right:
        if text[left] != text[right]:
            return False

        left += 1
        right -= 1

    return True
```

2.1 循环不变量

每轮开始时：

> `text[:left]` 与 `text[right+1:]` 已经完成配对并且全部匹配；只剩 `[left, right]` 尚未确认。

2.2 为什么条件是 `left < right`

- 奇数长度时，最终剩一个中间字符，不需要与自己比较；
- 偶数长度时，指针会相邻后交错；
- 当 `left >= right`，所有需要比较的字符对均已通过。

时间 `O(n)`，额外空间 `O(1)`。

---

3. 忽略非字母数字的回文串

题目可能要求忽略标点和大小写：

```
[python]
def is_valid_palindrome(text):
    left = 0
    right = len(text) - 1

    while left < right:
        while left < right and not text[left].isalnum():
            left += 1

        while left < right and not text[right].isalnum():
            right -= 1

        if text[left].lower() != text[right].lower():
            return False

        left += 1
        right -= 1

    return True
```

这里有三层循环，但总时间仍是 `O(n)`：每个指针只会单向移动，任何位置最多被各自经过一次。

---

4. 有序数组的两数之和

```
[python]
def two_sum_sorted(nums, target):
    left = 0
    right = len(nums) - 1

    while left < right:
        total = nums[left] + nums[right]

        if total == target:
            return [left, right]
        if total < target:
            left += 1
        else:
            right -= 1

    return []
```

4.1 为什么这样移动

数组已经升序排列。

- 和太小：只有增大较小数才可能达到目标，因此左指针右移；
- 和太大：只有减小较大数才可能达到目标，因此右指针左移。

4.2 与哈希表两数之和比较

方法	前提	时间	空间
哈希表	无需有序	`O(n)` 平均	`O(n)`
左右指针	数组有序	`O(n)`	`O(1)`

如果为了使用双指针先排序，要考虑排序 `O(n log n)`，并且原下标会变化。

---

5. 盛最多水的容器

两条竖线下标为 `left`、`right`，面积为：

```
   ×
(right - left) × min(height[left], height[right])
```

```
[python]
def max_area(height):
    left = 0
    right = len(height) - 1
    best = 0

    while left < right:
        width = right - left
        current_height = min(height[left], height[right])
        best = max(best, width * current_height)

        if height[left] <= height[right]:
            left += 1
        else:
            right -= 1

    return best
```

5.1 为什么移动较短的一侧

假设左边更短。保留左边并移动右边：

- 宽度一定变小；
- 高度仍然不可能超过左边这条短线的限制。

因此面积不可能变大，可以排除所有“左边不变、右边向内”的组合。

只有移动短边，才有机会遇到更高的边，弥补宽度下降。

这道题的关键不是公式，而是这个排除证明。

---

6. 读写指针：原地移动零

要求把所有零移动到末尾，同时保持非零元素相对顺序。

可以把 `write` 理解为：

> 下一个非零元素应该写入的位置。

```
[python]
def move_zeroes(nums):
    write = 0

    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write] = nums[read]
            write += 1

    while write < len(nums):
        nums[write] = 0
        write += 1
```

6.1 循环不变量

处理 `read` 之前：

```
[python]
nums[:write]
```

扫描结束后，前缀已经正确，只需把剩余位置填零。

6.2 交换写法

```
[python]
def move_zeroes(nums):
    slow = 0

    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
```

这种写法一步完成移动和填零。两种都正确，但必须理解 `slow` 的含义。

---

7. 删除有序数组中的重复项

有序数组中的相同元素相邻。保留每个值的第一个：

```
[python]
def remove_duplicates(nums):
```

```
    if not nums:
        return 0

    write = 1

    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]:
            nums[write] = nums[read]
            write += 1

    return write
```

返回 `write` 后，有效结果是：

```
[python]
nums[:write]
```

循环不变量：

> `nums[:write]` 是已处理前缀去重后的结果；`write` 是下一个新值的写入位置。

不要要求数组后半部分保持原值。原地题通常只规定前 `k` 个位置有效。

---

8. 两个有序数组的交集式扫描

两个指针分别指向两个有序序列：

```
[python]
def intersection_sorted(nums1, nums2):
    i = 0
    j = 0
    result = []

    while i < len(nums1) and j < len(nums2):
        if nums1[i] == nums2[j]:
            result.append(nums1[i])
            i += 1
            j += 1
        elif nums1[i] < nums2[j]:
            i += 1
        else:
            j += 1

    return result
```

较小元素不可能与对方当前或之后的更大元素相等，因此可以安全跳过。

这类模式也用于：

- 合并两个有序数组；
- 比较两个字符串处理结果；
- 求区间交集；
- 归并排序中的合并阶段。

---

9. 三数之和

题目：找出不重复的三元组，使和为 0。

暴力三层循环为 `O(n³)`。排序后，可以固定第一个数，再对剩余部分使用左右指针：

```
[python]
def three_sum(nums):
    nums.sort()
    result = []

    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        if nums[i] > 0:
            break

        left = i + 1
        right = len(nums) - 1

        while left < right:
            total = nums[i] + nums[left] + nums[right]

            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1

                while left < right and nums[left] == nums[left - 1]:
                    left += 1

                while left < right and nums[right] == nums[right + 1]:
                    right -= 1

    return result
```

9.1 去重分成两层

固定值去重：

```
if i > 0 and nums[i] == nums[i - 1]:
    continue
```

找到答案后，左右值去重：

```
[python]
while left < right and nums[left] == nums[left - 1]:
    left += 1
```

为什么不能一看到 `nums[left] == nums[left-1]` 就无条件跳过？

因为在尚未找到答案时，当前重复值可能仍需与不同的右值组合。去重必须与枚举逻辑对应。

9.2 复杂度

```
O(n log n)
i         O(n)   O(n)
 O(n²)
                       O(1)   O(log n)
```

---

10. 快慢指针的其他含义

第 6 讲链表已经见过：

- 快慢指针找中点；
- Floyd 算法检测环。

这里要统一认识：快慢指针不一定真的是“速度不同”。它们也可以职责不同：

```
fast / read
slow / write
```

做题时先描述职责，不要只写“定义两个指针”。

---

11. 双指针与滑动窗口的区别

它们都可能使用 `left` 和 `right`，但目的不同。

双指针

两个位置协作排除组合或原地整理数据：

滑动窗口

`left` 和 `right` 共同定义一个连续区间，并维护区间内部的统计信息：

k

滑动窗口是同向双指针的一个特殊模式，但它强调“窗口状态”。下一讲会专门学习。

---

12. 常见错误

12.1 没有利用有序性却强行双指针

无序两数之和不能根据和的大小决定移动方向。应使用哈希表，或先排序并处理原下标问题。

12.2 移错了指针

不能凭口诀移动。必须根据单调性解释为什么某一侧可以排除。

12.3 `left <= right` 与 `left < right` 混淆

两数配对不能使用同一元素两次时通常是 `left < right`。处理单个候选位置时可能需要 `<=`。

12.4 原地写入覆盖尚未读取的数据

读写指针中通常保证 `write <= read`，这样写入不会破坏未来数据。需要反向合并时，则从数组末尾向前写。

12.5 忘记结果只在前缀有效

删除重复项返回长度 `k`，答案是 `nums[:k]`，不是整个数组。

12.6 三数之和去重位置错误

固定数去重与左右指针去重是不同层次，不能混为一条判断。

12.7 指针没有保证前进

每条分支都要让至少一个指针发生变化，否则可能死循环。

---

13. 本讲检查表

```
1.
2.
3.
4.
5.
6.        left < right    left <= right
7.
8.
9.
10.
```

14. 本讲总结

1. 双指针利用顺序或单调性减少重复枚举。
2. 左右指针常用于有序数组、回文和区间收缩。
3. 读写指针常用于稳定地原地整理数组。
4. 两序列指针每次排除当前较小者，常用于合并和比较。
5. 三数之和是“排序 + 固定一个数 + 两数双指针”。
6. 不要背移动口诀；每次移动都要能够说明排除了哪些不可能情况。

第 12 讲：滑动窗口——维护一个连续区间的状态

0. 本讲的核心问题

题目经常要求在数组或字符串的连续部分中寻找答案：

- 长度为 `k` 的连续子数组最大和；
- 不含重复字符的最长子串；
- 和至少为目标值的最短子数组；
- 字符串中是否存在某种排列；
- 最小覆盖子串。

暴力方法通常枚举每个左端点、每个右端点，再重新计算区间内容，可能达到 `O(n²)` 或 `O(n³)`。

滑动窗口的核心思想是：

> 当区间边界只向前移动时，复用上一个区间的统计结果，而不是从头计算。

窗口通常表示半开区间：

```
[left, right]
```

在代码中也常把 `right` 作为当前刚加入的位置。

---

1. 什么题可能使用滑动窗口

看到以下信号时可以考虑：

1. 研究对象是连续子数组或连续子串；
2. 右端加入一个元素后，窗口状态能增量更新；
3. 左端删除一个元素后，窗口状态也能增量更新；
4. 存在某种单调性，使左端不需要回退。

不是所有连续区间题都能使用普通滑动窗口。

例如数组含负数时，“窗口和太大就缩小”可能不再安全，因为删除一个负数反而会使和增大。

---

2. 固定长度窗口

题目：求长度恰好为 `k` 的连续子数组最大和。

2.1 暴力方法

每个窗口重新求和：

```
[python]
def max_sum_fixed_brute(nums, k):
    best = float("-inf")
```

```
        for left in range(len(nums) - k + 1):
            current = sum(nums[left:left + k])
            best = max(best, current)

        return best
```

有约 `n` 个窗口，每次求和 `O(k)`：

```
O(nk)
```

2.2 复用前一个窗口

窗口向右移动一格：

```
   +
```

```
[python]
def max_sum_fixed(nums, k):
    if k <= 0 or k > len(nums):
        return None

    window_sum = sum(nums[:k])
    best = window_sum

    for right in range(k, len(nums)):
        entering = nums[right]
        leaving = nums[right - k]
        window_sum += entering - leaving
        best = max(best, window_sum)

    return best
```

时间降为 `O(n)`。

循环不变量：

> 更新后，`window_sum` 等于长度为 `k`、右端点为 `right` 的窗口元素之和。

---

3. 可变长度窗口的标准流程

可变窗口通常让右端不断扩张；当窗口不满足条件时，移动左端收缩。

```
[python]
left = 0

for right in range(len(data)):
    add(data[right])

    while window_is_invalid():
        remove(data[left])
        left += 1

    update_answer(left, right)
```

四步含义：

```
1. right
2.
3.         left
4.
```

3.1 为什么看似嵌套仍是 `O(n)`

`right` 从左到右最多移动 `n` 次；`left` 也只向右最多移动 `n` 次。

所有 `while` 的总执行次数最多为 `n`，因此通常是 `O(n)`，不是 `O(n²)`。

---

4. 核心题：无重复字符的最长子串

窗口内不允许出现重复字符。

4.1 使用集合

```
[python]
def length_of_longest_substring(text):
    left = 0
    window = set()
    best = 0

    for right, ch in enumerate(text):
        while ch in window:
            window.remove(text[left])
            left += 1

        window.add(ch)
        best = max(best, right - left + 1)

    return best
```

4.2 窗口不变量

在更新答案时：

> `window` 恰好包含 `text[left:right+1]` 中的字符，并且这些字符互不重复。

4.3 为什么必须使用 `while`

例如：

```
abba
```

加入第二个 `b` 时，可能需要连续移除多个左侧字符，直到旧 `b` 被移出窗口。

只写 `if` 不能保证窗口恢复合法。

---

5. 用最后出现位置直接跳跃

集合写法让 `left` 一步步移动。也可以记录每个字符最后出现的下标：

```
[python]
def length_of_longest_substring(text):
    last_index = {}
    left = 0
    best = 0

    for right, ch in enumerate(text):
        if ch in last_index and last_index[ch] >= left:
            left = last_index[ch] + 1

        last_index[ch] = right
        best = max(best, right - left + 1)

    return best
```

为什么要检查：

```
[python]
last_index[ch] >= left
```

因为字符的上一次出现可能已经在当前窗口左侧，不能让 `left` 倒退。

也可写成：

```
[python]
left = max(left, last_index[ch] + 1)
```

窗口左端必须单调向右。

---

6. 最短子数组：和至少为 target

假设 `nums` 中都是正整数。

当窗口和达到目标后，继续扩大只会更长；应该尽量从左侧缩小：

```
[python]
def min_subarray_len(target, nums):
    left = 0
    window_sum = 0
    best = float("inf")

    for right, x in enumerate(nums):
        window_sum += x

        while window_sum >= target:
            best = min(best, right - left + 1)
            window_sum -= nums[left]
            left += 1

    return 0 if best == float("inf") else best
```

6.1 为什么在收缩前更新答案

当前窗口已经满足条件，所以它是一个候选答案。记录后再删除左端，尝试得到更短的合法窗口。

6.2 为什么要求正数

正数保证：

- 扩张右端时，窗口和不会减小；
- 收缩左端时，窗口和不会增大。

这形成了单调性。

如果含负数，简单的左右滑动不能保证不漏答案，可能需要前缀和与单调队列等方法。

---

7. “最长合法”与“最短满足”的更新位置

这两类模板容易混淆。

7.1 最长合法窗口

先收缩到合法，再更新：

```
[python]
for right in range(n):
    add(right)

    while invalid():
```

```
            remove(left)
            left += 1

        best = max(best, right - left + 1)
```

例：最长无重复子串。

7.2 最短满足条件窗口

一旦满足，就在收缩过程中不断更新：

```
[python]
for right in range(n):
    add(right)

    while valid():
        best = min(best, right - left + 1)
        remove(left)
        left += 1
```

例：和至少为 target 的最短正数子数组。

不要只背代码，要先判断答案是在“合法状态”更新，还是在“满足状态下尽量收缩”时更新。

---

8. 固定长度频率窗口：找到所有字母异位词

给定 `text` 和较短字符串 `pattern`，找出 `text` 中所有与 `pattern` 字母频率相同的子串起点。

窗口长度固定为 `len(pattern)`。

```
[python]
def find_anagrams(text, pattern):
    if len(pattern) > len(text):
        return []

    need = {}
    window = {}

    for ch in pattern:
        need[ch] = need.get(ch, 0) + 1

    result = []
    left = 0

    for right, ch in enumerate(text):
        window[ch] = window.get(ch, 0) + 1

        if right - left + 1 > len(pattern):
            leaving = text[left]
            window[leaving] -= 1

            if window[leaving] == 0:
                del window[leaving]

            left += 1

        if window == need:
            result.append(left)

    return result
```

字典直接比较清晰，但每次比较可能与不同字符数量有关。字符集固定时，可以使用长度 26 的数组，或维护匹配种类数量进行优化。

---

9. 频率差与满足种类数

更复杂的字符串窗口通常维护：

```
need[ch]         ch
window[ch]          ch
formed
required
```

当某字符数量第一次达到要求：

```
[python]
if window[ch] == need[ch]:
    formed += 1
```

当移除字符导致它从满足变为不足：

```
[python]
if window[ch] == need[ch]:
    formed -= 1

window[ch] -= 1
```

注意减少 `formed` 的判断发生在计数减一之前。

窗口合法条件：

```
[python]
formed == required
```

这比每次完整比较两个字典更高效。

---

10. 进阶题：最小覆盖子串

目标：找到 `text` 中包含 `target` 所有字符及其数量的最短子串。

```
[python]
def min_window(text, target):
    if not target or len(target) > len(text):
        return ""

    need = {}

    for ch in target:
        need[ch] = need.get(ch, 0) + 1

    window = {}
    required = len(need)
    formed = 0
    left = 0
    best_length = float("inf")
    best_left = 0

    for right, ch in enumerate(text):
        window[ch] = window.get(ch, 0) + 1

        if ch in need and window[ch] == need[ch]:
            formed += 1

        while formed == required:
            length = right - left + 1

            if length < best_length:
                best_length = length
                best_left = left

            leaving = text[left]
```

```
                if leaving in need and window[leaving] == need[leaving]:
                    formed -= 1

                window[leaving] -= 1
                left += 1

        if best_length == float("inf"):
            return ""

        return text[best_left:best_left + best_length]
```

10.1 思考顺序

这是“最短满足条件”模板的典型应用。

---

11. 窗口状态应该保存什么

窗口不一定保存窗口元素本身，因为原数组已经保存了元素。额外状态通常是：

- 元素和 `window_sum`；
- 字符集合 `window`；
- 频率字典 `counts`；
- 不同元素数量 `distinct`；
- 最大值或最小值的候选结构；
- 满足要求的字符种类数 `formed`。

选择最小够用的状态。

例如只关心是否重复时，集合足够；关心重复次数时，需要频率字典。

---

12. 滑动窗口何时失效

12.1 缺少单调性

若数组含正负数，窗口和随扩张和收缩都可能增减。根据当前和无法唯一决定移动哪一侧。

12.2 研究对象不是连续区间

子序列允许跳过元素，通常不是普通滑动窗口问题。

12.3 左端需要回退

标准窗口要求左右边界只向前。如果正确算法需要让左端反复回退，通常需要其他方法。

12.4 无法快速更新状态

如果加入和删除一个元素后都无法高效维护条件，滑动窗口可能没有优势，或需要更高级的数据结构。

---

13. 常见错误

13.1 混淆子串、子数组和子序列

- 子串/子数组必须连续；
- 子序列可以跳过元素。

普通滑动窗口处理连续区间。

13.2 窗口长度少写或多写 1

闭区间 `[left, right]` 的长度是：

```
[python]
right - left + 1
```

13.3 忘记移除左端对状态的影响

移动 `left` 前要先从和、集合或计数中删除 `data[left]`。

13.4 应用 `if` 而不是 `while`

一次收缩不一定能恢复合法。只要条件仍不满足，就要继续收缩。

13.5 `left` 倒退

使用最后出现位置跳跃时：

```
[python]
left = max(left, last_index[ch] + 1)
```

13.6 在错误位置更新答案

先判断是“最长合法”还是“最短满足”，再决定答案更新在收缩前、收缩中还是收缩后。

13.7 删除计数为零的键遗漏

如果使用字典整体相等来判断，零计数键仍会影响字典比较，需要删除。

---

14. 本讲检查表

```
1.
2.
3. left right
4.
5.
6.
7.         left    if    while
8.
9.
10.
11.    set
12.      k
```

15. 本讲总结

1. 滑动窗口复用相邻连续区间的统计状态。
2. 固定窗口每次移除左端、加入右端。

3. 可变窗口通常由右端扩张，并用左端恢复合法性或寻找最短答案。
4. 两个指针各自只向前，因此嵌套 `while` 的总时间通常仍为 `O(n)`。
5. “最长合法”和“最短满足”有不同的答案更新位置。
6. 使用滑动窗口前必须确认连续性、可增量更新和边界移动的单调性。

第 13 讲：二分查找——利用单调性每次排除一半

0. 本讲的核心问题

在排好序的数组中查找一个目标，线性扫描需要最坏 `O(n)` 时间。

如果查看中间元素：

- 中间值太小，左半部分全部不可能；
- 中间值太大，右半部分全部不可能。

每次把搜索范围缩小约一半，这就是二分查找。

二分查找的本质不是“数组中间取一个数”，而是：

> 搜索空间具有单调性，因此一次判断能够安全排除一整半候选。

除了有序数组，二分还可以用于“答案范围”：判断某个答案是否可行，并利用可行性的单调变化寻找边界。

---

1. 为什么时间是 `O(log n)`

搜索范围长度变化：

```
n → n/2 → n/4 → n/8 → ... → 1
```

经过 `k` 次后：

```
n / 2^k ≈ 1
```

因此：

```
k ≈ log₂(n)
```

即使数组有一百万个元素，二分也只需要大约 20 次判断。

---

2. 最重要的选择：区间定义

二分最常见错误不是算法思想，而是区间边界不一致。

常用两套写法：

1. 闭区间 `[left, right]`；
2. 左闭右开 `[left, right)`。

两套都正确，但循环条件、更新方式必须与区间定义匹配。

本讲主要使用闭区间：

```
left     right
```

---

3. 标准二分：查找一个等于 target 的位置

```python
def binary_search(nums, target):
    left = 0
    right = len(nums) - 1

    while left <= right:
        middle = left + (right - left) // 2

        if nums[middle] == target:
            return middle
        elif nums[middle] < target:
            left = middle + 1
        else:
            right = middle - 1

    return -1
```

3.1 循环不变量

每轮开始时：

> 如果 target 存在，那么它一定在闭区间 `[left, right]` 中。

3.2 为什么循环条件是 `left <= right`

闭区间在 `left == right` 时仍包含一个候选元素，必须检查。

当 `left > right` 时，闭区间为空，说明目标不存在。

3.3 为什么更新要加减 1

`middle` 已经检查过且不等于目标，下一轮必须排除它：

```python
left = middle + 1
right = middle - 1
```

如果写成 `left = middle`，当区间只剩两个元素时可能永远不缩小，导致死循环。

---

4. 用表格手动跟踪

数组：

```
[1, 3, 5, 7, 9]
```

查找 `7`：

left	right	middle	nums[middle]	操作
0	4	2	5	太小，`left = 3`
3	4	3	7	找到

遇到二分边界问题时，不要凭感觉修改代码。选择：

- 空数组；
- 单元素；

- 两元素；
- 目标在首尾；
- 目标不存在；

逐轮写出 `left / middle / right`。

---

5. 左闭右开写法

了解另一套写法有助于阅读代码：

```
[python]
def binary_search_half_open(nums, target):
    left = 0
    right = len(nums)

    while left < right:
        middle = left + (right - left) // 2

        if nums[middle] == target:
            return middle
        elif nums[middle] < target:
            left = middle + 1
        else:
            right = middle

    return -1
```

因为右端不包含在搜索范围中，所以排除中点及右侧时可以写：

```
[python]
right = middle
```

学习阶段建议选定一套主模板，不要在同一道题里混用。

---

6. 查找边界：第一个大于等于 target 的位置

数组可能有重复值：

```
[1, 2, 2, 2, 5]
```

普通二分找到任意一个 `2` 就返回，但有时需要第一个 `2`。

更通用的定义是：

> 找到第一个满足 `nums[i] >= target` 的位置。

这叫 lower bound。

```
[python]
def lower_bound(nums, target):
    left = 0
    right = len(nums)

    while left < right:
        middle = left + (right - left) // 2

        if nums[middle] < target:
            left = middle + 1
        else:
            right = middle
```

```
    return left
```

这里使用左闭右开区间 `[left, right)`，并允许答案等于 `len(nums)`，表示所有元素都小于 target。

6.1 为什么遇到等于时不能返回

当前 `middle` 虽然满足条件，但左侧可能还有更早的满足位置。应保留 `middle`，继续搜索左半部分：

```
[python]
right = middle
```

6.2 搜索插入位置

`lower_bound` 的结果正是把 target 插入有序数组且保持有序的位置：

```
[python]
def search_insert(nums, target):
    return lower_bound(nums, target)
```

---

7. 查找第一个大于 target 的位置

定义：

> 找到第一个满足 `nums[i] > target` 的位置。

```
[python]
def upper_bound(nums, target):
    left = 0
    right = len(nums)

    while left < right:
        middle = left + (right - left) // 2

        if nums[middle] <= target:
            left = middle + 1
        else:
            right = middle

    return left
```

重复值区间可以由：

```
[python]
first = lower_bound(nums, target)
after_last = upper_bound(nums, target)
```

得到。

如果 `first == after_last`，说明 target 不存在。

最后一个等于 target 的下标是：

```
[python]
after_last - 1
```

前提是先确认 target 存在。

---

8. 边界二分的统一理解：False 到 True

把每个位置是否满足条件写成布尔值。

例如查找第一个 `>= target`：

```
nums:       [1, 2, 2, 4, 7]
>= target:   F  F  F  T  T
```

问题变成：

> 找到单调布尔序列中第一个 True。

通用模板：

```
[python]
def first_true(left, right, condition):
    #    [left, right)        right
    while left < right:
        middle = left + (right - left) // 2

        if condition(middle):
            right = middle
        else:
            left = middle + 1

    return left
```

这个模板是“答案二分”的基础。

---

9. 旋转排序数组

原有序数组：

```
[0, 1, 2, 4, 5, 6, 7]
```

旋转后：

```
[4, 5, 6, 7, 0, 1, 2]
```

整体不再有序，但每次取中点后，至少有一半仍然有序。

```
[python]
def search_rotated(nums, target):
    left = 0
    right = len(nums) - 1

    while left <= right:
        middle = left + (right - left) // 2

        if nums[middle] == target:
            return middle

        if nums[left] <= nums[middle]:
            #
            if nums[left] <= target < nums[middle]:
                right = middle - 1
            else:
                left = middle + 1
        else:
            #
            if nums[middle] < target <= nums[right]:
                left = middle + 1
            else:
                right = middle - 1

    return -1
```

思考顺序：

```
1.
```

```
2. target
3.        target
```

这个版本通常假设元素互不重复。大量重复值会使“哪一半有序”的判断失去区分度，需要额外处理。

---

10. 寻找旋转数组最小值

无重复元素时，可以与右端值比较：

```
[python]
def find_min_rotated(nums):
    left = 0
    right = len(nums) - 1

    while left < right:
        middle = left + (right - left) // 2

        if nums[middle] > nums[right]:
            left = middle + 1
        else:
            right = middle

    return nums[left]
```

如果 `nums[middle] > nums[right]`，最小值一定在 middle 右侧；否则 middle 可能就是最小值，所以保留它并令 `right = middle`。

注意这里的目标是边界，不是寻找某个等于值，因此循环和更新与标准二分不同。

---

11. 二维矩阵二分

若矩阵每行有序，并且下一行首元素大于上一行末元素，可以把它看成一个长度为 `rows * cols` 的虚拟一维数组。

一维下标 `middle` 映射到：

```
[python]
row = middle // cols
col = middle % cols
```

```
[python]
def search_matrix(matrix, target):
    if not matrix or not matrix[0]:
        return False

    rows = len(matrix)
    cols = len(matrix[0])
    left = 0
    right = rows * cols - 1

    while left <= right:
        middle = left + (right - left) // 2
        row = middle // cols
        col = middle % cols
        value = matrix[row][col]

        if value == target:
            return True
        if value < target:
            left = middle + 1
        else:
            right = middle - 1

    return False
```

不需要真的创建扁平数组。

---

12. 在答案范围上二分

有些题没有给出可直接搜索的有序数组，但答案存在一个数值范围，并且“某个答案是否可行”具有单调性。

例如：在限定天数内运送包裹，求最小运载能力。

若能力为 `capacity`：

- 太小可能无法按时完成；
- 某能力一旦可行，更大的能力也都可行。

形成：

```
F  F  F  T  T  T
```

目标是第一个 True。

12.1 解题步骤

```
1.
2.    can_finish(answer)
3.
4.
```

---

13. 示例：在 D 天内运送包裹

每天按顺序装载，不能改变包裹顺序。求最低运载能力。

答案边界：

```
[python]
left = max(weights)      #
right = sum(weights)     #
```

判断某个能力需要多少天：

```
[python]
def days_needed(weights, capacity):
    days = 1
    current_load = 0

    for weight in weights:
        if current_load + weight > capacity:
            days += 1
            current_load = 0

        current_load += weight

    return days
```

二分最小可行能力：

```
[python]
def ship_within_days(weights, days):
    left = max(weights)
    right = sum(weights)

    while left < right:
        middle = left + (right - left) // 2

        if days_needed(weights, middle) <= days:
```

```
                right = middle
            else:
                left = middle + 1

        return left
```

13.1 为什么可行时保留 middle

`middle` 已经可行，但它可能不是最小可行值，因此令：

```
[python]
right = middle
```

继续向左寻找边界。

总复杂度：

```
 O(n)
 O(log(answer_range))
O(n log(answer_range))
```

---

14. 怎样识别答案二分

看到“最大化最小值”或“最小化最大值”时可以考虑，但不能只靠关键词。

真正需要确认：

1. 答案可以在一个有序范围中枚举；
2. 给定候选答案，可以较快判断是否可行；
3. 可行性是单调的。

常见问题：

- 最小运载能力；
- 最低吃香蕉速度；
- 分配任务的最小最大负载；
- 能放置物体的最大最小距离。

如果可行性不是单调的，不能直接二分答案。

---

15. 常见错误

15.1 混用区间定义

先在纸上写：

```
[left, right]   [left, right)
```

再决定循环条件和更新。

15.2 搜索范围不缩小

闭区间标准查找必须排除 middle：

```
[python]
left = middle + 1
right = middle - 1
```

15.3 忽略单元素区间

闭区间使用 `while left <= right`。写成 `<` 会漏掉最后一个候选。

15.4 找边界时遇到等于就返回

若要求第一个或最后一个位置，等于只说明当前是候选，还要继续向边界搜索。

15.5 答案范围上下界错误

答案二分要先证明边界：左端可能不可行，右端通常应保证可行；或者使用明确的 first-true 区间约定。

15.6 `can_finish` 不具有单调性

先写出一串 False/True 示意，再证明只会发生一次转变。

15.7 空输入直接访问首尾

在读取 `nums[0]`、`matrix[0]` 前处理题目约束或空输入。

---

16. 本讲检查表

```
1.
2.
3.
4. left right
5.
6. middle
7.               /
8. condition(middle)
9.
10.
```

17. 本讲总结

1. 二分依赖单调性，每次判断排除约一半候选。
2. 区间定义必须与循环条件、边界更新保持一致。
3. 标准闭区间查找使用 `left <= right` 和 `middle ± 1`。
4. 边界问题可统一为寻找单调布尔序列中的第一个 True。
5. 旋转数组每轮至少有一半有序，可以据此排除。
6. 答案二分需要“范围 + 判断函数 + 单调性证明”。

第 14 讲：回溯——在选择树中尝试、撤销与继续

0. 本讲的核心问题

有些题要求找出所有可能方案：

- 所有子集；
- 所有排列；
- 所有满足条件的组合；
- 棋盘上的合法放置方式；
- 单词能否沿网格路径拼出。

暴力枚举的思路可能并没有错，但需要一种系统方法保证：

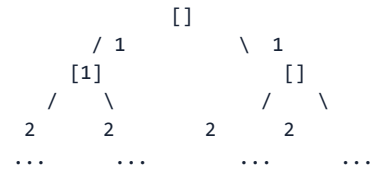
1. 每个候选方案都被考虑；
2. 不会遗漏或无意义重复；
3. 发现当前路径不可能成功时尽早停止；
4. 尝试完一个选择后，状态能恢复，再尝试其他选择。

回溯就是：

> 在决策树中进行深度优先搜索；做出选择，递归探索，返回后撤销选择。

1. 决策树视角

以 `nums = [1, 2, 3]` 的子集为例。对每个数都有两个选择：选或不选。



从根到叶子的一条路径代表一个完整决定。

回溯代码通常维护：

```
path
choices
end condition
```

2. 回溯的通用骨架

```python
result = []
path = []

def backtrack(state):
    if is_complete(state):
        result.append(path.copy())
        return

    for choice in available_choices(state):
        make_choice(choice)
        backtrack(next_state)
        undo_choice(choice)
```

最关键的三步：

撤销后，程序回到做选择之前的状态，才能正确尝试下一个分支。

---

3. 为什么必须复制 path

错误写法：

```python
result.append(path)
```

`path` 是同一个可变列表对象。后续 `append` 和 `pop` 会修改它，`result` 中此前保存的引用也会看到变化。

正确写法：

```python
result.append(path.copy())
```

或：

```python
result.append(path[:])
```

保存的是当前内容的副本。

这与链表章节的引用语义相同：变量保存对象引用，不会自动复制对象。

---

4. 子集：每个节点都是答案

对于不重复数组，使用 `start` 表示下一次只能从哪个下标之后选择。

```python
def subsets(nums):
    result = []
    path = []

    def backtrack(start):
        result.append(path.copy())

        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
```

```
        return result
```

4.1 为什么进入函数就记录答案

子集可以是任意长度，包括空集。决策树中的每个节点代表一个合法子集，不需要等到固定深度。

4.2 `start` 的含义

> `start` 之前的元素已经决定是否选择；下一次只能选择下标不小于 `start` 的元素。

使用 `i + 1` 进入下一层，保证：

- 同一个元素不被重复选择；
- `[1, 2]` 与 `[2, 2]` 不会作为两个子集重复出现。

4.3 复杂度

`n` 个元素有 `2^n` 个子集。仅输出就需要指数级规模：

```
O(n × 2^n)  #      path
O(n)        #           path
```

指数复杂度不是一定写错；当题目要求输出所有组合时，答案数量本身就是指数级。

---

5. 组合：选择固定数量的元素

从 `1..n` 中选择 `k` 个数：

```
[python]
def combine(n, k):
    result = []
    path = []

    def backtrack(start):
        if len(path) == k:
            result.append(path.copy())
            return

        for value in range(start, n + 1):
            path.append(value)
            backtrack(value + 1)
            path.pop()

    backtrack(1)
    return result
```

与子集的区别：

- 子集：每个长度都合法，进入函数就记录；
- 组合：只有长度等于 `k` 才记录并返回。

---

6. 组合剪枝

假设还需要选择：

```
[python]
need = k - len(path)
```

如果从 `value` 到 `n` 剩余数字不足 `need` 个，继续循环没有意义。

最大的起始选择为：

```
n - need + 1
```

因此：

```
[python]
for value in range(start, n - need + 2):
    ...
```

剪枝不是改变答案，而是提前停止不可能完成的分支。

剪枝前必须能证明：

> 被跳过的分支一定无法产生合法答案。

---

7. 排列：每层可以选择所有未使用元素

排列中顺序不同算不同答案：

```
[1, 2]   [2, 1]
```

因此不能使用只向右的 `start`。每层要从所有未使用元素中选择。

```
[python]
def permutations(nums):
    result = []
    path = []
    used = [False] * len(nums)

    def backtrack():
        if len(path) == len(nums):
            result.append(path.copy())
            return

        for i in range(len(nums)):
            if used[i]:
                continue

            used[i] = True
            path.append(nums[i])

            backtrack()

            path.pop()
            used[i] = False

    backtrack()
    return result
```

7.1 `used` 的含义

`used[i]` 表示下标 `i` 的元素是否已经出现在当前路径中。

它不是全局永久访问标记。递归返回后必须恢复为 `False`，让其他排列分支能够使用该元素。

7.2 子集、组合、排列的区别

问题	下一层候选	何时记录
子集	当前下标之后	每个节点
固定组合	当前下标之后	长度达到 k
排列	所有未使用元素	长度达到 n

---

8. 组合总和：一个元素能否重复选择

候选数可以重复使用，求和等于 target 的组合：

```
[python]
def combination_sum(candidates, target):
    result = []
    path = []

    def backtrack(start, remaining):
        if remaining == 0:
            result.append(path.copy())
            return

        if remaining < 0:
            return

        for i in range(start, len(candidates)):
            value = candidates[i]
            path.append(value)
            backtrack(i, remaining - value)
            path.pop()

    backtrack(0, target)
    return result
```

这个写法假设候选数都是正数。若包含 `0` 或负数，`remaining` 不一定持续减小，搜索可能无法终止；必须根据题目增加使用次数限制或改用其他状态设计。

注意递归传入的是 `i`，不是 `i + 1`：

```
[python]
backtrack(i, remaining - value)
```

因为当前元素允许再次选择。

若每个元素只能使用一次，则传 `i + 1`。

做题前必须写清：

---

9. 排序与去重：含重复元素的子集

输入：

```
[1, 2, 2]
```

如果直接按普通子集搜索，会多次产生 `[1, 2]`。

先排序，让相同值相邻；同一递归层中跳过相同选择：

```
[python]
def subsets_with_duplicates(nums):
    nums.sort()
    result = []
    path = []

    def backtrack(start):
        result.append(path.copy())

        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue

            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
    return result
```

9.1 为什么条件是 `i > start`

它只跳过同一层中的重复选择。

不同层仍可选择相同值。例如输入有两个 `2` 时，子集 `[2, 2]` 应保留：

- 第一层选择第一个 `2`；
- 下一层选择第二个 `2`。

如果写成 `i > 0`，会错误禁止不同层使用重复值。

---

10. 含重复元素的全排列

同样先排序，再在同一层避免选取等价的重复元素：

```
[python]
def unique_permutations(nums):
    nums.sort()
    result = []
    path = []
    used = [False] * len(nums)

    def backtrack():
        if len(path) == len(nums):
            result.append(path.copy())
            return

        for i in range(len(nums)):
            if used[i]:
                continue

            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue

            used[i] = True
            path.append(nums[i])

            backtrack()

            path.pop()
            used[i] = False

    backtrack()
    return result
```

条件：

```
[python]
nums[i] == nums[i - 1] and not used[i - 1]
```

表示同一层中，前一个相同值还没有被使用时，不允许跳过它去选后一个相同值，从而规定一种统一顺序。

这条规则较难，不要只背。画出 `[1, 1, 2]` 的前两层决策树观察重复分支。

---

11. 括号生成

生成 `n` 对合法括号。任意前缀必须满足：

≤

```
[python]
def generate_parentheses(n):
    result = []
    path = []

    def backtrack(open_count, close_count):
        if len(path) == 2 * n:
            result.append("".join(path))
            return

        if open_count < n:
            path.append("(")
            backtrack(open_count + 1, close_count)
            path.pop()

        if close_count < open_count:
            path.append(")")
            backtrack(open_count, close_count + 1)
            path.pop()

    backtrack(0, 0)
    return result
```

这道题展示了剪枝的价值：非法前缀根本不进入搜索树。

---

12. 网格回溯：单词搜索

题目：单词能否由网格中上下左右相邻字符组成，同一格不能重复使用。

```
[python]
def word_exists(board, word):
    if not word:
        return True
    if not board or not board[0]:
        return False

    rows = len(board)
    cols = len(board[0])
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    def backtrack(row, col, index):
        if board[row][col] != word[index]:
            return False

        if index == len(word) - 1:
            return True

        original = board[row][col]
        board[row][col] = "#"
```

```
                for dr, dc in directions:
                    nr = row + dr
                    nc = col + dc

                    if 0 <= nr < rows and 0 <= nc < cols:
                        if board[nr][nc] != "#":
                            if backtrack(nr, nc, index + 1):
                                board[row][col] = original
                                return True

                board[row][col] = original
                return False

            for row in range(rows):
                for col in range(cols):
                    if backtrack(row, col, 0):
                        return True

            return False
```

12.1 状态含义

```
(row, col)
index        word[index]
board    #
```

12.2 为什么返回 True 前也要恢复

如果题目允许修改输入且找到后立即结束，技术上可能不影响返回结果。但保持“任何函数返回前恢复现场”更安全，也让函数没有意外副作用。

12.3 visited 为什么是路径级状态

某个格子不能在同一条路径中重复使用，但可以在另一条候选路径中使用。因此进入时标记，离开时撤销。

这与普通图遍历的全局 `visited` 不同。

---

13. 回溯与普通 DFS 的区别

两者都使用深度优先搜索，但访问标记的生命周期常不同。

普通图 DFS

回溯

判断依据：

> 这个状态是“全局已经处理”，还是“当前方案暂时使用”？

---

14. 返回布尔值还是收集所有答案

14.1 找到一个答案即可

```
def backtrack(...):
    if goal:
        return True

    for choice in choices:
        make(choice)

        if backtrack(...):
            undo(choice)
            return True

        undo(choice)

    return False
```

14.2 收集所有答案

```
def backtrack(...):
    if goal:
        result.append(path.copy())
        return

    for choice in choices:
        make(choice)
        backtrack(...)
        undo(choice)
```

先确定题目要一个解、判断存在，还是全部解。返回设计会不同。

---

15. 怎样把思路写成回溯代码

编码前填写：

```
1. path
2.
3.
4.
5.
6.
7.
8.
9.
10.
```

如果这些问题还没有答案，不要急着写模板。

---

16. 常见错误

16.1 忘记撤销选择

`append` 后通常要有对应的 `pop`；修改棋盘后要恢复原值；设置 `used[i] = True` 后要恢复 `False`。

16.2 把 path 本身加入结果

必须复制：

```
[python]
result.append(path.copy())
```

16.3 `start`、`i`、`i+1` 混淆

- 不允许重复选当前元素：下一层传 `i + 1`；
- 允许重复选：下一层传 `i`；
- 排列：通常不用 start，而用 used。

16.4 结束条件记录后忘记 `return`

固定长度答案形成后通常应返回，否则还会继续添加元素。

16.5 去重条件跨错层级

组合/子集常用：

```
[python]
i > start and nums[i] == nums[i - 1]
```

强调“同一递归层”。

16.6 剪枝没有正确性依据

不能因为“感觉不会成功”就跳过。剪枝必须由剩余数量、排序后的大小关系或约束条件证明。

16.7 全局 `visited` 与路径 `visited` 混淆

回溯路径中的占用状态通常需要撤销。

---

17. 复杂度怎样分析

回溯复杂度通常由决策树规模决定：

```
^
```

但不同问题的精确答案数量不同：

- 子集：`2^n` 个结果；
- 排列：`n!` 个结果；
- 固定组合：`C(n, k)` 个结果。

还要考虑复制答案的成本。

回溯优化的主要方向不是把指数复杂度变成多项式，而是：

- 不进入必然失败的分支；
- 避免生成重复答案；
- 尽早返回；
- 用约束缩小候选集合。

---

18. 本讲检查表

```
1.
2. path
3.
4.
5.      path.copy()
6.
7.
8.      i    i+1
9.      used
10.
11.
12.
```

19. 本讲总结

1. 回溯是对选择树进行 DFS：选择、递归、撤销。
2. `path` 表示当前方案，保存答案时必须复制。
3. 子集和组合用 start 控制选择顺序；排列用 used 记录当前路径占用。
4. 传 `i` 表示可以重复使用当前元素，传 `i+1` 表示不能重复使用。
5. 排序后在同一层跳过相同值，可以去除重复答案。
6. 普通 DFS 的 visited 常是全局的；回溯占用状态通常属于当前路径，离开时要恢复。
7. 剪枝必须有正确性依据，而不是凭感觉跳过。

第 15 讲：动态规划入门——保存重复子问题的答案

0. 本讲的核心问题

有些问题可以分解成更小的相同类型问题：

- 到第 `i` 级台阶有多少种走法？
- 偷到第 `i` 间房屋时最多得到多少钱？
- 凑出金额 `a` 最少需要多少枚硬币？
- 到网格位置 `(row, col)` 有多少条路径？

直接递归可能反复计算同一个子问题。

动态规划 DP 的核心是：

> 明确定义子问题的状态，保存其答案，并按照依赖顺序组合出更大问题的答案。

动态规划不是“看到最值就套数组”，而是一套推导过程：

→ → → →

---

1. 从斐波那契数列看重复计算

递归定义：

```
[python]
def fibonacci(n):
    if n <= 1:
        return n

    return fibonacci(n - 1) + fibonacci(n - 2)
```

计算 `fibonacci(5)` 时：

```
fib(5)
├── fib(4)
│   ├── fib(3)
│   └── fib(2)
└── fib(3)
    ├── fib(2)
    └── fib(1)
```

`fib(3)`、`fib(2)` 被反复计算，形成指数级调用。

两个特征提示动态规划：

1. 重叠子问题：相同状态被多次求解；
2. 最优子结构或可组合结构：大问题答案能由小问题答案得到。

---

2. 两种实现方式

2.1 自顶向下：记忆化搜索

仍按递归思路求解，但把计算结果保存起来：

```
[python]
def fibonacci(n):
    memo = {}

    def dp(x):
        if x <= 1:
            return x

        if x in memo:
            return memo[x]

        memo[x] = dp(x - 1) + dp(x - 2)
        return memo[x]

    return dp(n)
```

也可以使用：

```
[python]
from functools import cache

@cache
def fibonacci(n):
    if n <= 1:
        return n

    return fibonacci(n - 1) + fibonacci(n - 2)
```

记忆化搜索适合先写出自然递归关系。

2.2 自底向上：表格递推

先算小状态，再算大状态：

```
[python]
def fibonacci(n):
    if n <= 1:
        return n

    dp = [0] * (n + 1)
    dp[0] = 0
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

自底向上避免递归深度问题，计算顺序更明确。

---

3. DP 五步法

遇到动态规划题，先写以下五项，再编码。

第一步：定义状态

用一句完整的话定义：

```
dp[i]
```

状态定义必须包含范围和约束。例如：

```
dp[i]       i
```

不要只写“dp[i] 是答案”。

第二步：写状态转移

问最后一步从哪里来：

```
i
```

第三步：确定初始状态

转移无法继续向前推时，需要哪些已知值？

第四步：确定计算顺序

计算 `dp[i]` 时，它依赖的状态必须已经完成。

第五步：确定返回值

答案一定是 `dp[-1]` 吗？有时是最大值、最小值、整行或多个状态的组合。

---

4. 爬楼梯

每次可以走 1 或 2 级，问到第 `n` 级有多少种方法。

4.1 状态定义

```
dp[i]     i
```

4.2 最后一步分析

到达第 `i` 级的最后一步只有两种可能：

- 从 `i-1` 走 1 级；
- 从 `i-2` 走 2 级。

两类方案互不重叠：

```
dp[i] = dp[i-1] + dp[i-2]
```

4.3 初始状态

```
dp[1] = 1
dp[2] = 2
```

4.4 代码

```
[python]
def climb_stairs(n):
    if n <= 2:
        return n

    dp = [0] * (n + 1)
    dp[1] = 1
    dp[2] = 2

    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

时间 `O(n)`，空间 `O(n)`。

---

5. 空间优化：只保存真正需要的状态

`dp[i]` 只依赖前两个状态，不需要保存整张表：

```
[python]
def climb_stairs(n):
    if n <= 2:
        return n

    two_steps_before = 1
    one_step_before = 2

    for i in range(3, n + 1):
        current = one_step_before + two_steps_before
        two_steps_before = one_step_before
        one_step_before = current

    return one_step_before
```

空间降为 `O(1)`。

学习初期不要过早压缩。先用数组写清状态，确认正确后再观察依赖范围。

---

6. 打家劫舍：状态必须包含约束

相邻房屋不能同时偷，求最大金额。

6.1 状态定义

一种简洁定义：

```
dp[i]      i
```

注意这里的 `i` 是房屋数量，数组中第 `i` 间房屋下标是 `i-1`。

6.2 最后一个决定

考虑第 `i` 间房屋：

- 不偷：答案是 `dp[i-1]`；
- 偷：上一间不能偷，答案是 `dp[i-2] + nums[i-1]`。

取最大：

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
```

6.3 初始状态

```
dp[0] = 0
dp[1] = nums[0]
```

6.4 代码

```
[python]
def rob(nums):
    n = len(nums)

    if n == 0:
        return 0

    dp = [0] * (n + 1)
    dp[0] = 0
    dp[1] = nums[0]

    for i in range(2, n + 1):
        skip = dp[i - 1]
        take = dp[i - 2] + nums[i - 1]
        dp[i] = max(skip, take)

    return dp[n]
```

6.5 不变量

计算完 `dp[i]` 后：

> `dp[i]` 是前 `i` 间房屋所有合法选择中的最大金额。

状态转移完整枚举了最后一间“偷”或“不偷”两种互斥情况。

---

7. 最大子数组和

求连续子数组的最大和。

7.1 状态定义必须包含“以 i 结尾”

```
dp[i]     nums[i]
```

为什么不是“前 i 个数中的最大和”？

为了向右延伸连续区间，需要知道当前最优区间是否确实接在 `i` 位置。

7.2 转移

以 `i` 结尾的子数组：

- 从 `nums[i]` 重新开始；
- 接在以 `i-1` 结尾的最优子数组后。

```
dp[i] = max(nums[i], dp[i-1] + nums[i])
```

7.3 答案不是 `dp[-1]`

最大子数组可能在数组中间结束，因此答案是：

```
max(dp)
```

空间优化写法：

```
[python]
def max_subarray(nums):
    current = nums[0]
    best = nums[0]
```

```
    for i in range(1, len(nums)):
        current = max(nums[i], current + nums[i])
        best = max(best, current)

    return best
```

这里 `current` 就是压缩后的 `dp[i]`。

---

8. 不同路径：二维 DP

机器人从左上角到右下角，只能向右或向下。

8.1 状态定义

```
dp[row][col]      (row, col)
```

8.2 转移

到达当前格子的最后一步来自：

- 上方 `(row-1, col)`；
- 左方 `(row, col-1)`。

```
dp[row][col] = dp[row-1][col] + dp[row][col-1]
```

8.3 初始状态

第一行只能一直向右，第一列只能一直向下，因此全部为 1。

```
[python]
def unique_paths(rows, cols):
    dp = [[1] * cols for _ in range(rows)]

    for row in range(1, rows):
        for col in range(1, cols):
            dp[row][col] = dp[row - 1][col] + dp[row][col - 1]

    return dp[rows - 1][cols - 1]
```

复杂度：

```
O(rows × cols)
O(rows × cols)
```

8.4 一维压缩

当前行只依赖上一行和当前行左侧：

```
[python]
def unique_paths(rows, cols):
    dp = [1] * cols

    for row in range(1, rows):
        for col in range(1, cols):
            dp[col] = dp[col] + dp[col - 1]

    return dp[-1]
```

更新前 `dp[col]` 是上方路径数；`dp[col-1]` 已更新，是左方路径数。

---

9. 最小路径和

每个格子有代价，只能向右或向下，求最小路径和。

状态：

```
dp[row][col]
```

转移：

```
dp[row][col] = grid[row][col] + min(
    dp[row-1][col],
    dp[row][col-1]
)
```

```
[python]
def min_path_sum(grid):
    rows = len(grid)
    cols = len(grid[0])
    dp = [[0] * cols for _ in range(rows)]

    dp[0][0] = grid[0][0]

    for col in range(1, cols):
        dp[0][col] = dp[0][col - 1] + grid[0][col]

    for row in range(1, rows):
        dp[row][0] = dp[row - 1][0] + grid[row][0]

    for row in range(1, rows):
        for col in range(1, cols):
            dp[row][col] = grid[row][col] + min(
                dp[row - 1][col],
                dp[row][col - 1],
            )

    return dp[-1][-1]
```

二维 DP 最容易错在第一行和第一列的边界初始化。

也可以给表格外侧设置哨兵值，但学习初期分开初始化更清楚。

---

10. 零钱兑换：完全背包入门

给定硬币面值，每种可无限使用，求凑成 amount 的最少硬币数。

10.1 状态定义

```
dp[value]      value
```

10.2 转移

如果最后使用一枚面值为 `coin` 的硬币，那么之前要凑出：

```
value - coin
```

所以：

```
dp[value] = min(dp[value], dp[value-coin] + 1)
```

10.3 初始值与“不可能”状态

```
dp[0] = 0
```

其他金额一开始未知，可以设置为一个不可能优于答案的大值：

```
[python]
amount + 1
```

因为最多使用 `amount` 枚面值 1 的硬币；若连这都做不到，大值仍保留。

10.4 代码

```
[python]
def coin_change(coins, amount):
    impossible = amount + 1
    dp = [impossible] * (amount + 1)
    dp[0] = 0

    for value in range(1, amount + 1):
        for coin in coins:
            if coin <= value:
                dp[value] = min(
                    dp[value],
                    dp[value - coin] + 1,
                )

    if dp[amount] == impossible:
        return -1

    return dp[amount]
```

复杂度：

```
O(amount × len(coins))
O(amount)
```

---

11. 为什么贪心选择最大硬币不一定正确

硬币：

```
[1, 3, 4]
```

金额 `6`：

- 贪心：先选 4，再选 1、1，共 3 枚；
- 最优：3 + 3，共 2 枚。

局部最大的选择不保证全局最优。DP 会比较所有可能的最后一枚硬币，同时复用更小金额的最优答案。

使用贪心前需要证明贪心选择性质，不能只因为它“看起来最快”。

---

12. 最长递增子序列

子序列不要求连续。定义：

```
dp[i]   nums[i]
```

要接到 `nums[i]` 前面的元素必须满足：

```
j < i   nums[j] < nums[i]
```

```
[python]
def length_of_lis(nums):
    if not nums:
        return 0

    dp = [1] * len(nums)

    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)
```

为什么初始值是 1?

每个元素单独都能组成长度为 1 的递增子序列。

为什么答案是 `max(dp)` 而不是 `dp[-1]`?

最长递增子序列不一定以最后一个元素结束。

这个基础解法时间 `O(n²)`。还有 `O(n log n)` 解法，但入门阶段应先掌握状态定义。

---

13. 0/1 背包的核心结构

有若干物品，每个物品只能选一次；容量有限，求最大价值。

二维定义：

```
dp[i][capacity]      i         capacity
```

对第 `i` 个物品：

- 不选它；
- 容量足够时选它。

```
dp[i][c] = max(
    dp[i-1][c],
    dp[i-1][c-weight] + value
)
```

13.1 一维压缩为什么要倒序

```
[python]
def knapsack_01(weights, values, capacity):
    dp = [0] * (capacity + 1)

    for weight, value in zip(weights, values):
        for current_capacity in range(capacity, weight - 1, -1):
            dp[current_capacity] = max(
                dp[current_capacity],
                dp[current_capacity - weight] + value,
            )

    return dp[capacity]
```

容量倒序保证右侧的：

```
[python]
dp[current_capacity - weight]
```

仍来自处理当前物品之前的上一轮，因此当前物品不会被重复使用。

如果正序更新，同一物品可能在一轮中被使用多次，那会变成完全背包的语义。

这一点先理解，不要求现在记住所有背包变体。

---

14. 记忆化搜索与表格 DP 如何选择

更适合记忆化搜索

- 递归关系容易自然描述；
- 状态空间中只有一部分真正会被访问；
- 计算顺序难以直接确定；

- 需要从回溯逐步优化。

更适合表格 DP

- 状态范围规则；
- 依赖顺序明确；
- 希望避免递归深度；
- 需要进行空间压缩。

二者本质相同：都保证每个状态只计算有限次，并保存结果。

---

15. DP 与回溯、贪心的区别

回溯

枚举选择路径，常用于列出全部方案。状态通常在返回时撤销。

动态规划

把不同路径汇聚到同一状态，只保存这个状态所需的答案，不再关心完整选择历史。

贪心

每步只做当前看起来最优的选择，不回头，也不比较所有子问题。需要单独证明局部选择能导向全局最优。

例如打家劫舍中，不能简单总偷金额更大的相邻房屋；DP 比较“偷当前”和“不偷当前”两类完整子问题。

---

16. 怎样判断状态是否定义充分

一个好状态必须满足：

> 只知道状态中保存的信息，就足以计算未来，不需要重新查看被丢弃的历史细节。

例如股票问题中，如果只定义：

```
dp[i] =   i
```

可能不够，因为未来卖出时还需要知道当前是否持有股票。应增加状态维度：

```
dp[i][0]   i
dp[i][1]   i
```

状态不够时，不要硬写转移；先补充缺失信息。

---

17. 正确性怎样解释

动态规划的正确性通常来自两部分：

17.1 状态覆盖所有合法情况

例如打家劫舍的最后一间只有“偷”和“不偷”两种，互斥且完整。

17.2 转移使用的子问题已经最优

如果 `dp[i-2]` 不是前 `i-2` 间房的最优答案，那么用它构造的结果也无法保证最优。

这通常可以用数学归纳法理解：

1. 初始状态正确；
2. 假设更小状态正确；
3. 转移完整比较所有最后一步，因此当前状态正确。

面试时不必写正式证明，但应该能说出“最后一步有哪些互斥情况”。

---

18. 常见错误

18.1 状态定义模糊

必须写清 `dp[i]` 是否包含第 `i` 个元素、是否必须以 `i` 结尾、是否只考虑前 `i` 个元素。

18.2 下标与数量混淆

`dp[i]` 表示前 `i` 个元素时，当前数组元素通常是 `nums[i-1]`。

18.3 初始状态不符合定义

不要先抄 `dp[0] = 0`。从状态定义推导空集合、第一项应是多少。

18.4 计算顺序错误

使用某状态前，它必须已经计算完成。二维表可能需要从左上到右下，也可能相反。

18.5 返回错位置

最大子数组和、最长递增子序列返回 `max(dp)`，不一定是 `dp[-1]`。

18.6 最小值问题把未知状态初始化为 0

0 可能被错误当成更优解。最小值问题常用 `float("inf")` 或可靠的大值。

18.7 空间压缩后覆盖依赖

先画出当前状态依赖上一行还是当前行，再决定正序或倒序。

18.8 看见递归就加缓存

缓存键必须包含决定子问题答案的全部状态。如果遗漏变量，不同子问题会错误共享结果。

---

19. 本讲检查表

```
1.
2.
3. dp[i]   dp[i][j]
4.
5.
6.
7.
8.
9.
10.
11.      O(n)       O(1)
12.
```

20. 本讲总结

1. 动态规划通过保存子问题答案避免重复计算。
2. 标准推导顺序是状态、转移、初始化、计算顺序和返回值。
3. 记忆化搜索与自底向上 DP 本质相同，只是求解顺序不同。
4. 状态定义必须包含未来决策所需的全部信息。
5. “最后一步”分析常能自然推出状态转移。
6. 空间优化应在正确的一维或二维 DP 之后进行。
7. DP 的复杂度通常是“状态数量 × 每个状态的转移成本”。

---

21. 学完 15 讲后的训练顺序

完成本讲后，不要立刻追求刷完 Hot 100。建议按模式各选代表题达到能够独立完成：

```
/

/BFS

        BST
  k     k
```

每个模式先让一两道代表题达到 C 级，再扩展到外观不同的题。真正目标是看到新题时能够说出：